

# 图像中的目标检测

李宇楠

计算机科学与技术学院

2025/12/5



1

目标检测的任务及挑战

2

目标检测模型

3

检测结果评估

4

检测应用

5

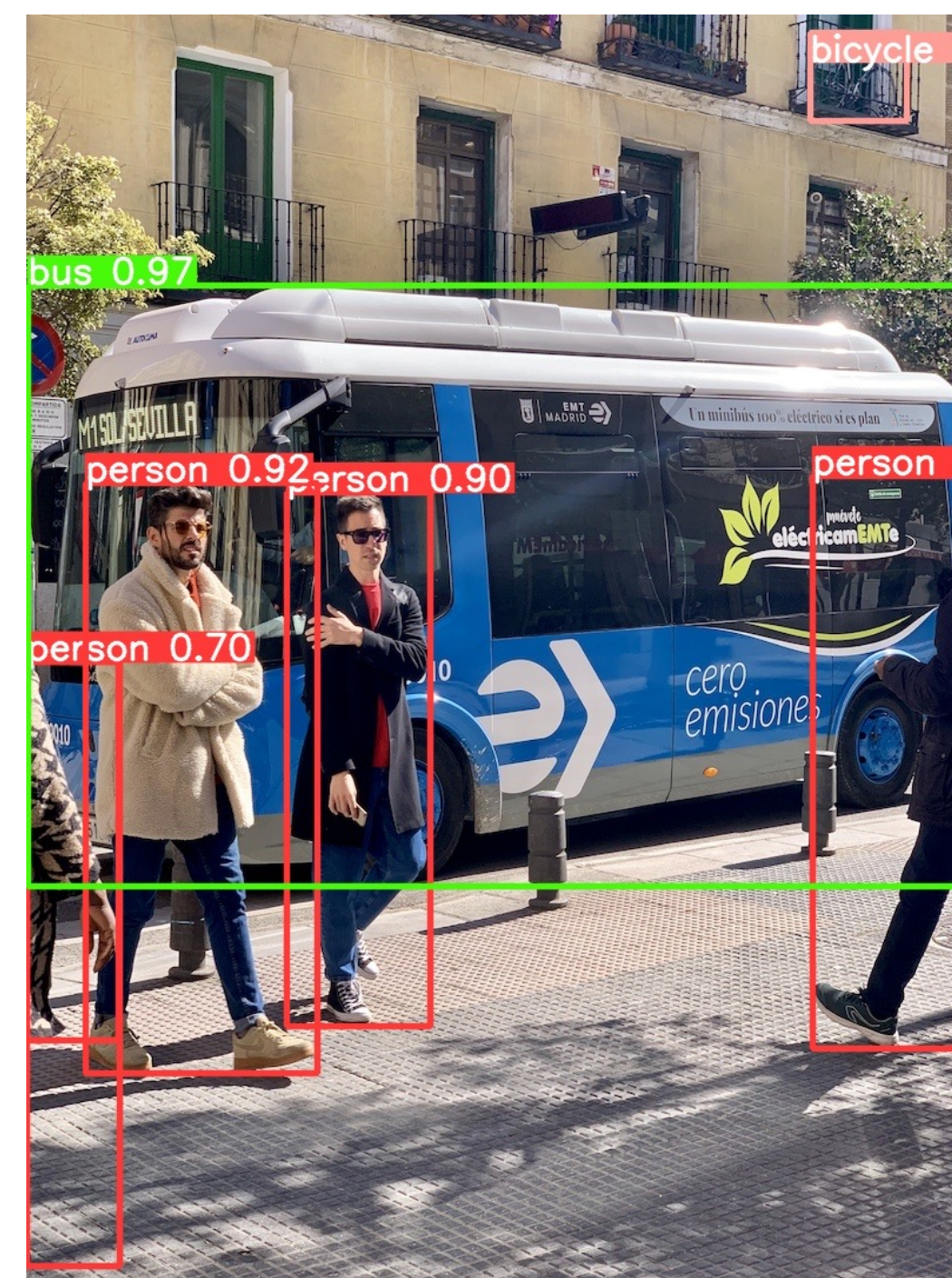
目标检测数据集



# 一、目标检测的任务及挑战

## 目标检测:

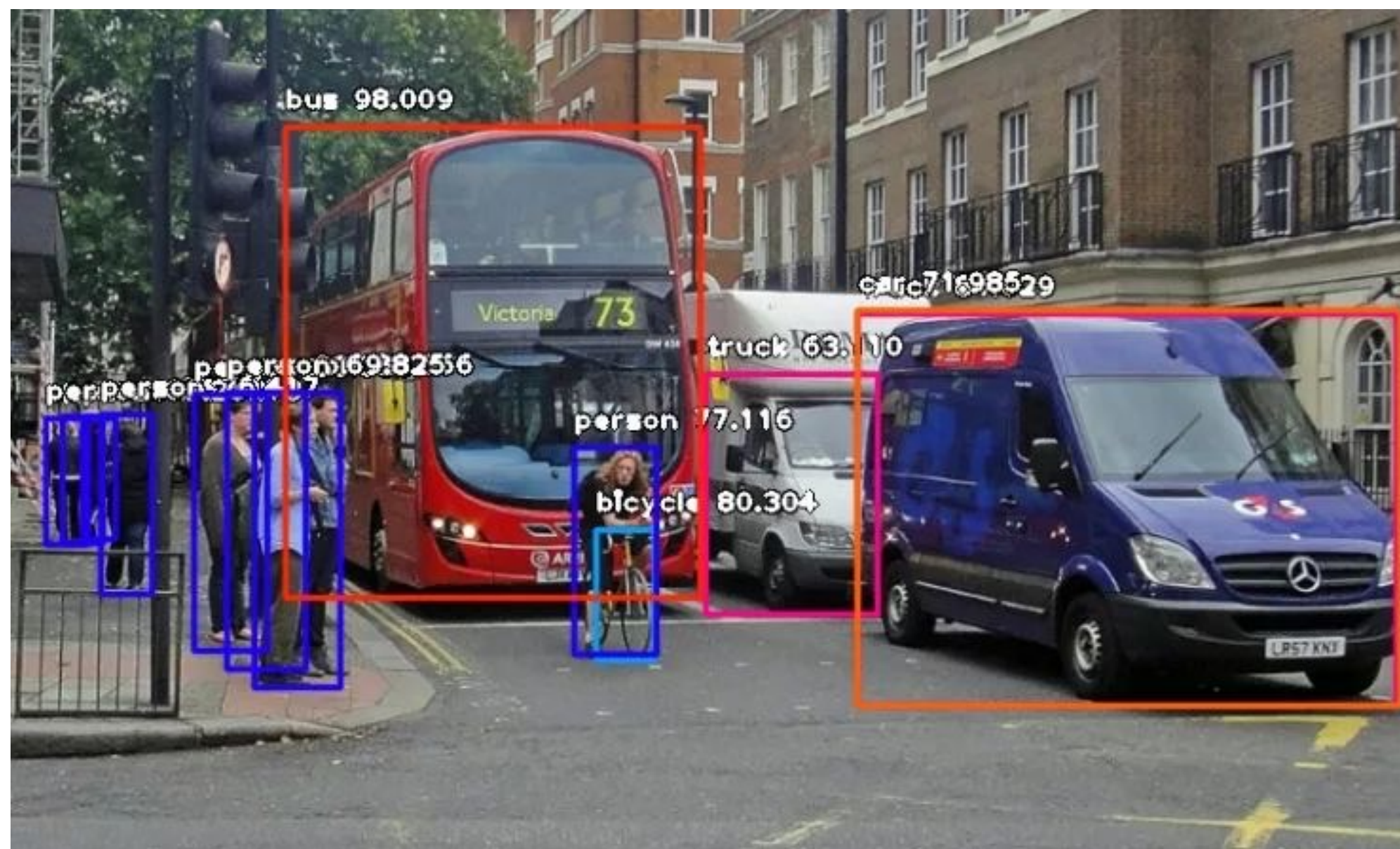
计算机视觉的核心任务之一，是一个找出图像中所有感兴趣的目标（物体），确定他们的类别和位置的问题





# 一、目标检测的任务及挑战

## 目标检测



- 环境变化
- 遮挡
- 表征差异
- 多重长宽比
- 类别不均衡
- .....



## 二、目标检测模型

### 传统检测器

- Viola-Jones ( VJ )  
检测器——首次实现了对人脸的实时检测
- DPM——传统对象检测方法的顶峰

### 基于CNN的两级检测器

- RCNN——首次将深度卷积网络用于目标检测
- SPPNet——克服RCNN冗余特征计算速度慢问题
- Fast RCNN——集成了R-CNN和SPPNet的优点
- Faster RCNN检测器——第一个端到端和第一个近实时深度学习检测器
- FPN——在检测各种尺度的物体方面显示出了巨大的进步

### 基于CNN的一级检测器

- YOLO——深度学习时代的第一个单阶段检测器
- SSD——显着提高了一级检测器的检测精度 ( YOLO弱项 )
- RetinaNet——使一级检测器可以达到两级检测器相当的精度

### 基于Transformer的检测器

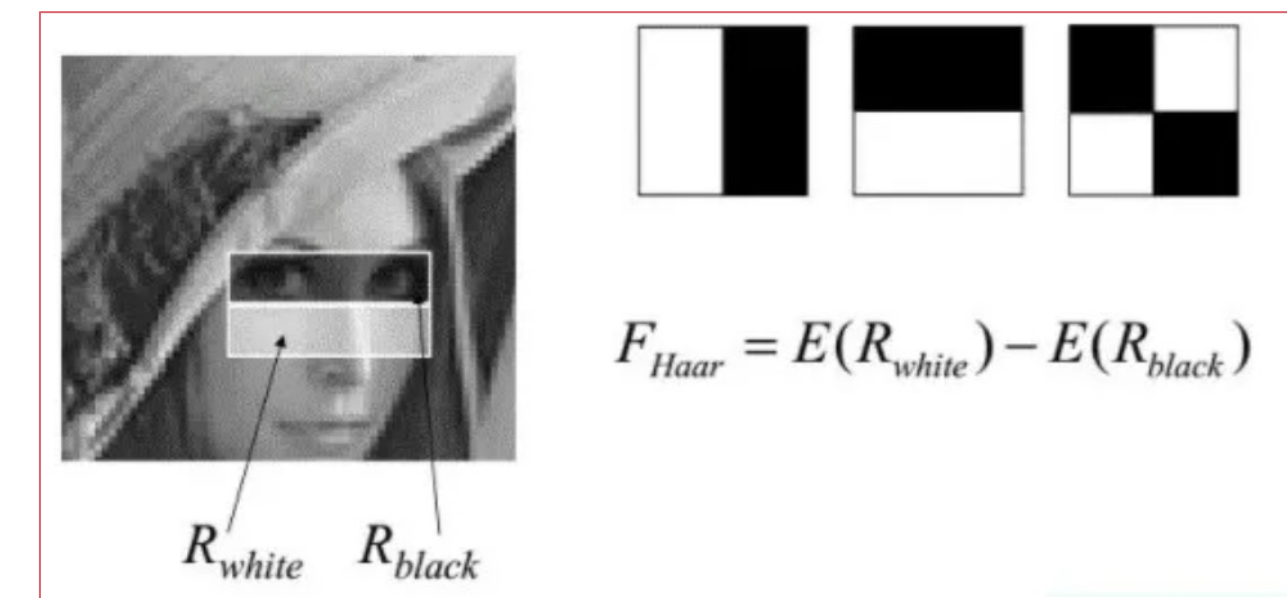
- DETR——基于Transformer的端到端的目标检测，不使用NMS、没有anchor等先验知识和约束

## 二、目标检测模型

### VJ检测器

- P. Viola和M. Jones在没有任何约束条件(如肤色分割)的情况下首次实现了人脸的实时检测,在同等的检测精度下,检测器的速度是其他算法的数十倍甚至数百倍。

2001年Paul Viola和Michael Jones基于Haar-like特征提出了一种快速且有效的人脸检测方法,如图所示。检测器采用滑动窗口的方式以检查目标是否存在窗口之中,该检测器看起来似乎很简单稳定,但由于计算量庞大导致时间复杂度极高。VJ检测器结合了“积分图像”、“特征选择”和“检测级联”三种重要技术,大大提高了检测速度。

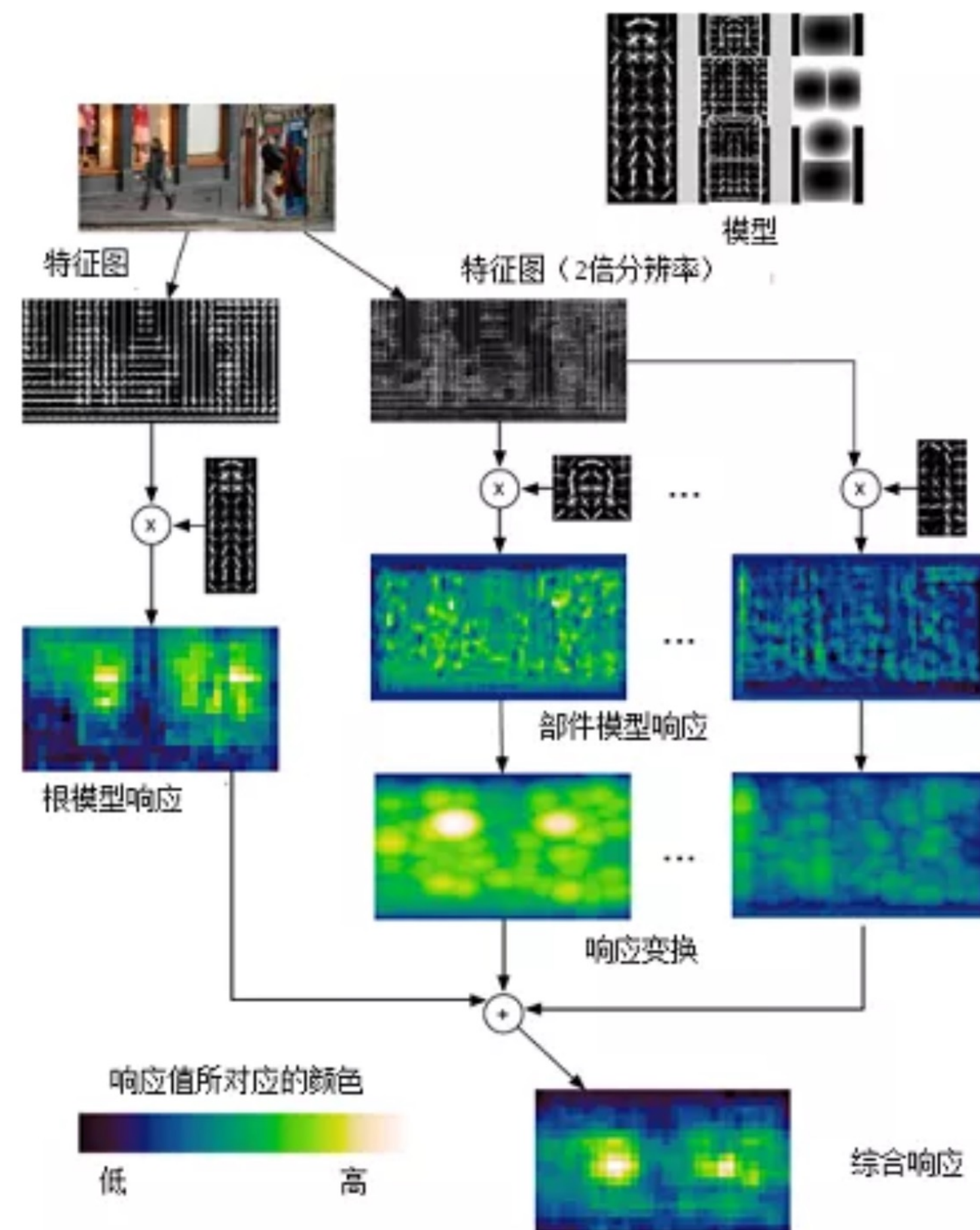


- 特征选择：作者没有使用一组手动选择的Haar基滤波器,而是使用Adaboost算法从一组巨大的随机特征池(大约180k维)中选择一组对人脸检测最有帮助的小特征。
- 检测级联：在VJ检测器中引入了一个多级检测范例(又称“检测级联”, detection cascades),通过减少对背景窗口的计算,而增加对人脸目标的计算,从而减少了计算开销。
- 积分图像：积分图像是一种计算方法,以加快盒滤波或卷积过程。与当时的其他目标检测算法一样,在VJ检测器中使用Haar小波作为图像的特征表示。积分图像使得VJ检测器中每个窗口的计算复杂度与其窗口大小无关。

## 二、目标检测模型

### DPM检测器

- DPM遵循“分而治之”的检测思想，训练可以简单地看作是学习一种正确的分解对象的方法，推理可以看作是对不同对象部件的检测的集合。
- DPM算法采用了改进后的HOG特征，SVM分类器和滑动窗口(Sliding Windows)检测思想，针对目标的多视角问题，采用了多组件(Component)的策略，针对目标本身的形变问题，采用了基于图结构(Pictorial Structure)的部件模型策略。
- 通过Hog特征模板来刻画每一部分，然后进行匹配。并且采用了金字塔，即在不同的分辨率上提取Hog特征。
- 利用提出的可变形组件模型，在进行检测时，detect window的得分等于part的匹配得分减去模型变化的花费。
- 在训练模型时，需要训练得到每一个part的HOG模板，以及衡量part位置分布cost的参数。文章中提出了LatentSVM方法，将deformable part model的学习问题转换为一个分类问题：利用SVM学习，将part的位置分布作为latent values，模型的参数转化为SVM的分割超平面。具体实现中，作者采用了迭代计算的方法，不断地更新模型。





## 二、目标检测模型

### 数据增强方案



原图



水平翻转



随机裁剪



垂直翻转



随机仿射



随机遮挡



伽马校正



随机噪声



网格掩码



颜色抖动



透视变换



随机擦除



# 二、目标检测模型

## 数据增强方案



Mixup



Cutmix



Mosaic

原图

## 二、目标检测模型

### 基于CNN的两级检测器-RCNN

- 在深度学习时代，目标检测可以分为两类：
- “ two-stage detection ” 和 “ one-stage detection ” ，前者将检测框定为一个 “ 从粗到细 ” 的过程，而后者将其定义为 “ 一步完成 ” 。
- 两级的RCNN背后的想法很简单：
- 第一步，生成候选区
- 第二步，判断每个候选区的类别
  - 用CNN提取特征
  - 用SVM分类
  - 用线性回归矫正候选框的位置

整体四个步骤，精度达到了当时水平的巅峰，但是每一张图片都会有2000左右的候选框，再加上SVM和线性回归的推理，一张图片的推理时间需要47s。

Object detection: Region-based CNN (RCNN)



**【selective search 选择性搜索】** 这个**选择性搜索**就是给出候选框的算法的名字。用Selective Search方法可以对一张图片生成大约1000~3000个候选框，具体的算法逻辑如下：

- 1.使用一种过分割手段，将图像分割成过小的区域；
- 2.查看现有小区域，合并可能性最高的两个区域，重复直到整张图像合并成一个区域位置；；
- 3.输出所有曾经存在过的区域，也就是从小合并到大的所有出现过的大大小小的候选框，即所谓候选区域。

**【合并遵守的原则】**

- 1.颜色（颜色直方图）相近的；
- 2.纹理（梯度直方图）相近的；
- 3.合并后总面积小的；

**把图片都缩放成统一的227\*227的大小**，然后把统一大小的图片输入到CNN中，进行特征提取。

**采用one vs all的SVM进行分类（训练5个SVM）**

训练一个线性回归模型，来做一个边框回归，精细的调整候选框（x,y,w,h）四个参数。

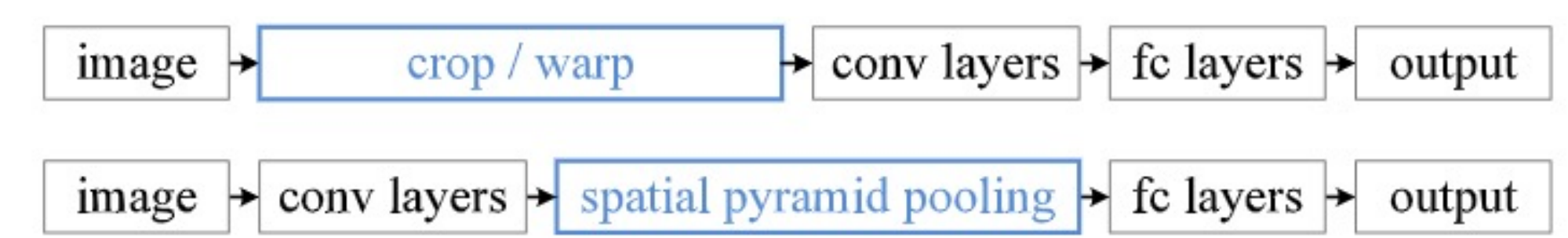
## 二、目标检测模型

### 基于CNN的两级检测器-SPP Net

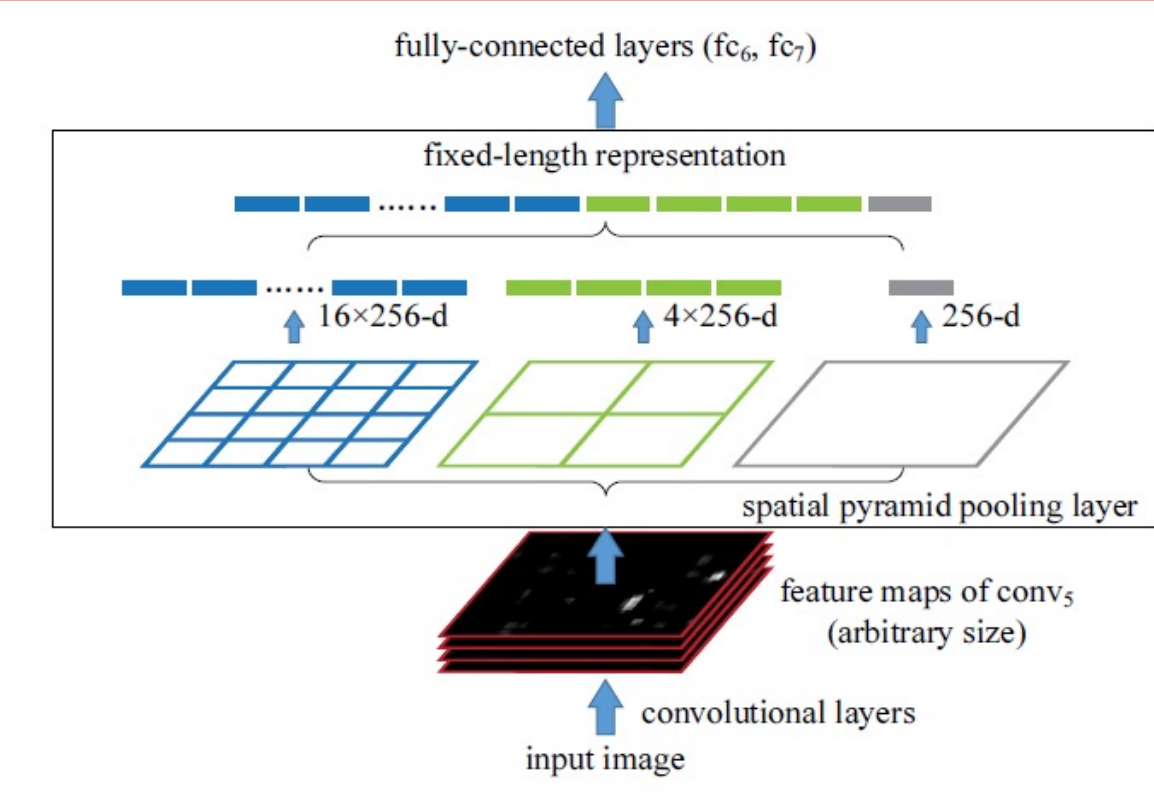
- 利用SPPNet进行目标检测时，只对  
整个图像进行一次特征映射计算，然  
后生成任意区域的定长表示，训练检  
测器，避免了卷积特征的重复计算。  
SPPNet的速度是R-CNN的20多倍，  
并且没有牺牲任何检测精度。
- 除此之外，避免检测过程中了crop和  
warp操作导致的图像变形、物体不  
全



【如何避免使用warp和crop】SPP net使用了SPP空间金字塔池化层，来实现了这个功能。先看和RCNN的模型结构对比：



RCNN中先通过warp将大小不同的候选框变成227统一大小输入到conv layers；SPP中直接对大小不同的候选框做conv layers，所以我们得到了大小不同的特征图。然后将**大小不同的特征图输入到SPP空间金字塔池化层中，输出变成大小相同的特征图**



He K , Zhang X , Ren S , et al. Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition[J]. IEEE Transactions on Pattern Analysis & Machine Intelligence, 2014, 37(9):1904–16.

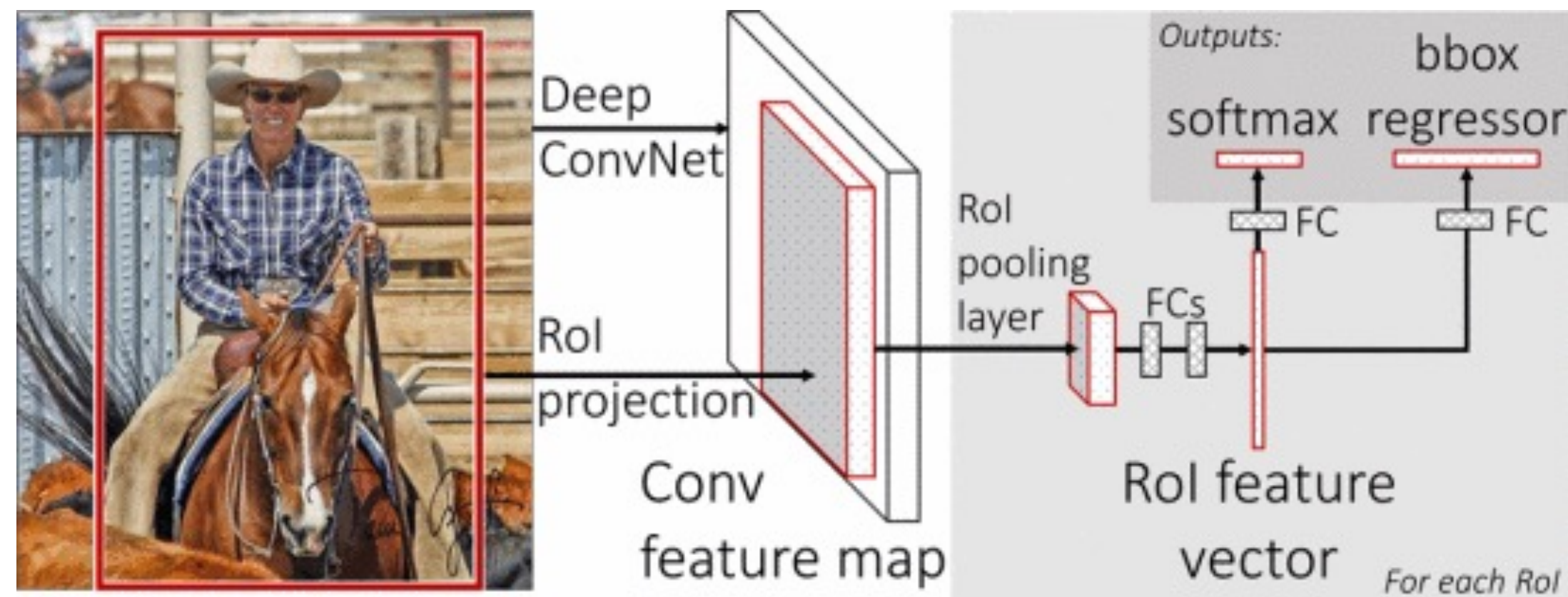
## 二、目标检测模型

### 基于CNN的两级检测器-Fast R-CNN

- R-CNN的进阶版Fast R-CNN就是在R-CNN的基础上采纳了SPP Net方法，对R-CNN作了改进，使得性能进一步提高。

1. 依旧使用selective search的方法来选取候选框bounding box。

2. 不再使用SVM+线性回归的方法，而是全部通过神经网络来实现分类和边框回归的任务。Fast-RCNN很重要的一个贡献是成功的让人们看到了Region Proposal + CNN这一框架的可行性，不再使用Region Proposal + CNN + SVM + LR的框架了。



1. 有一个原图，先通过selective search得到候选框；
2. 将原图放到CNN中进行特征处理，得到特征图，然后将候选框映射到特征图上，得到大小不同的特征图的候选框。
3. 把特征图候选框放到ROI pooling层中（只有一个，不是SPP的3个ROI结构了），输出7\*7的特征图，有512个通道，所以在ROI输出之后，把7\*7\*512个特征放到FC层中。
4. FC层后接入了两个不同的FC层（分支结构），分别输出两个不同的结果。第一个结果加上softmax变成候选框的类别概率，第二个结果就是候选框的4个参数的边框回归的值。

## 二、目标检测模型

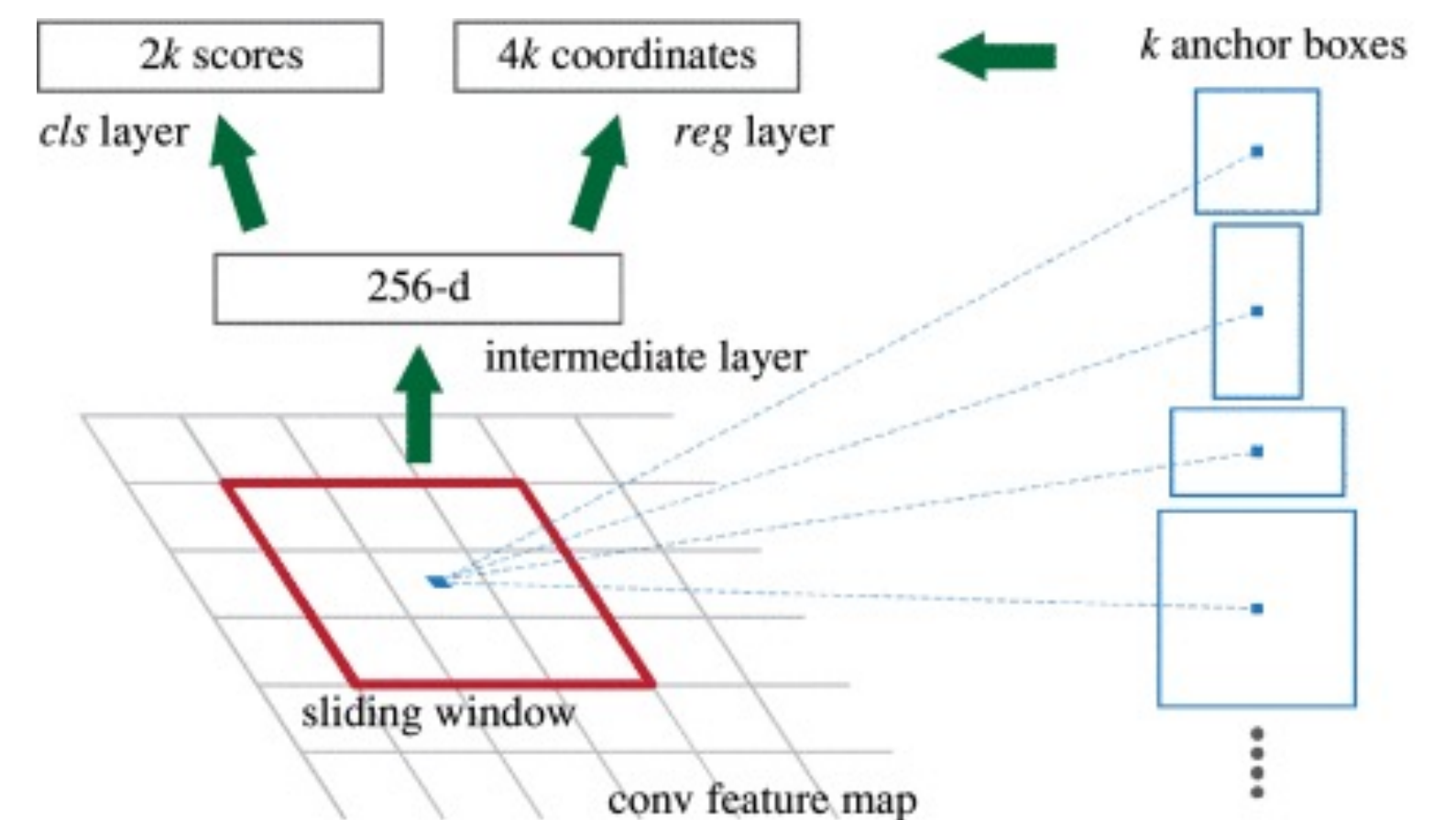
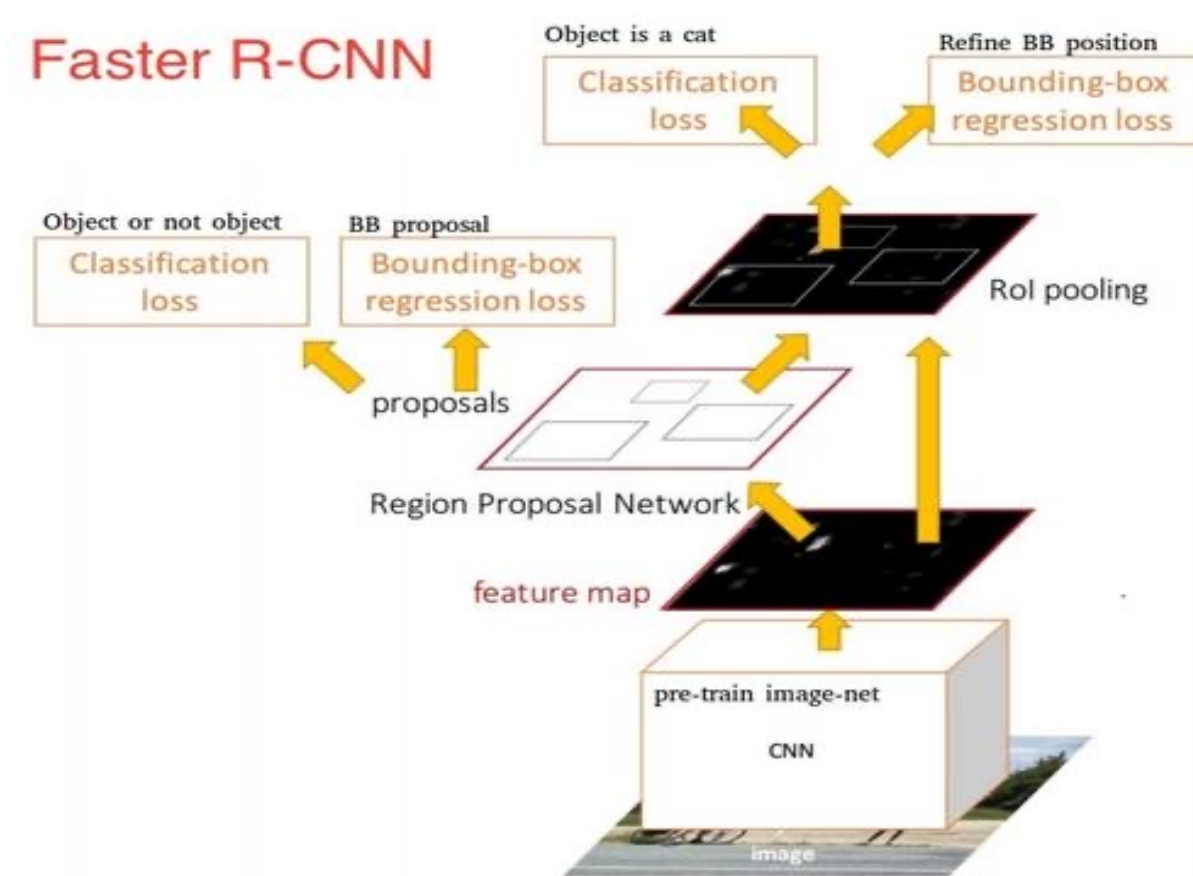
### 基于CNN的两级检测器-Faster R-CNN

- Fast-RCNN 成功地融合了 R-CNN 和 SPPNet 的优点，但其检测速度仍然受到选择性搜索的限制。然后，一个问题自然而然地出现了：“我们能用CNN模型生成对象建议吗？”
- Faster RCNN 是第一个端到端的，也是第一个接近实时的深度学习检测器。Faster RCNN 的主要贡献是引入了区域建议网络 (RPN)，使几乎 cost-free 的区域建议成为可能。

**【区域候选网络RPN】** Faster RCNN最大的贡献在一引入了Region Proposal Network ( RPN ) 网络来代替selective search。

RPN网络直接放在最后一个卷积层的后面，通过一个滑动窗口，在feature map上滑动。每一个滑动窗口会给出一个置信度，置信度低的可以理解为这个窗口是没有目标的，置信度高的再考虑不同类别的概率。

**【Anchor box先验框】** 因为滑动窗口一般是正方形的，但是实际的目标检测可能是长方形。也许比较近似正方形的长方形可以通过边框回归矫正，但是其他的长方形物体就非常难办了。这里事先设置了几种不同形状大小的候选框，除了正方形之外，还有长方形，小正方形等，这样虽然成倍的增加了计算量，但是可以提高准确度。而且这样增加候选框的计算量消耗依然是小于selective search的。



整个网络有四个损失函数：

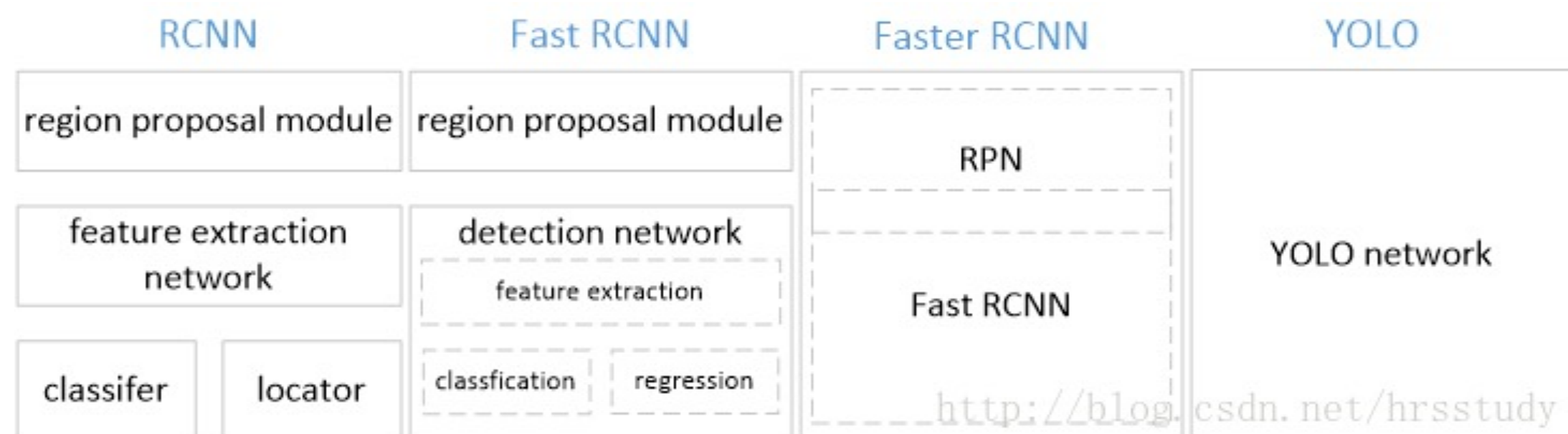
- RPN的二分类损失函数
- RPN的边框回归损失函数
- FAST RCNN的多分类损失函数
- Fast RCNN的边框回归损失函数

图中有四种不同的anchor box，意味着滑动到某一个位置的时候，以那个点为中心，产生四个不同大小的候选框，然后预测得到 是否是目标物体二分类结果 和 边框回归四分类结果。

## 二、目标检测模型

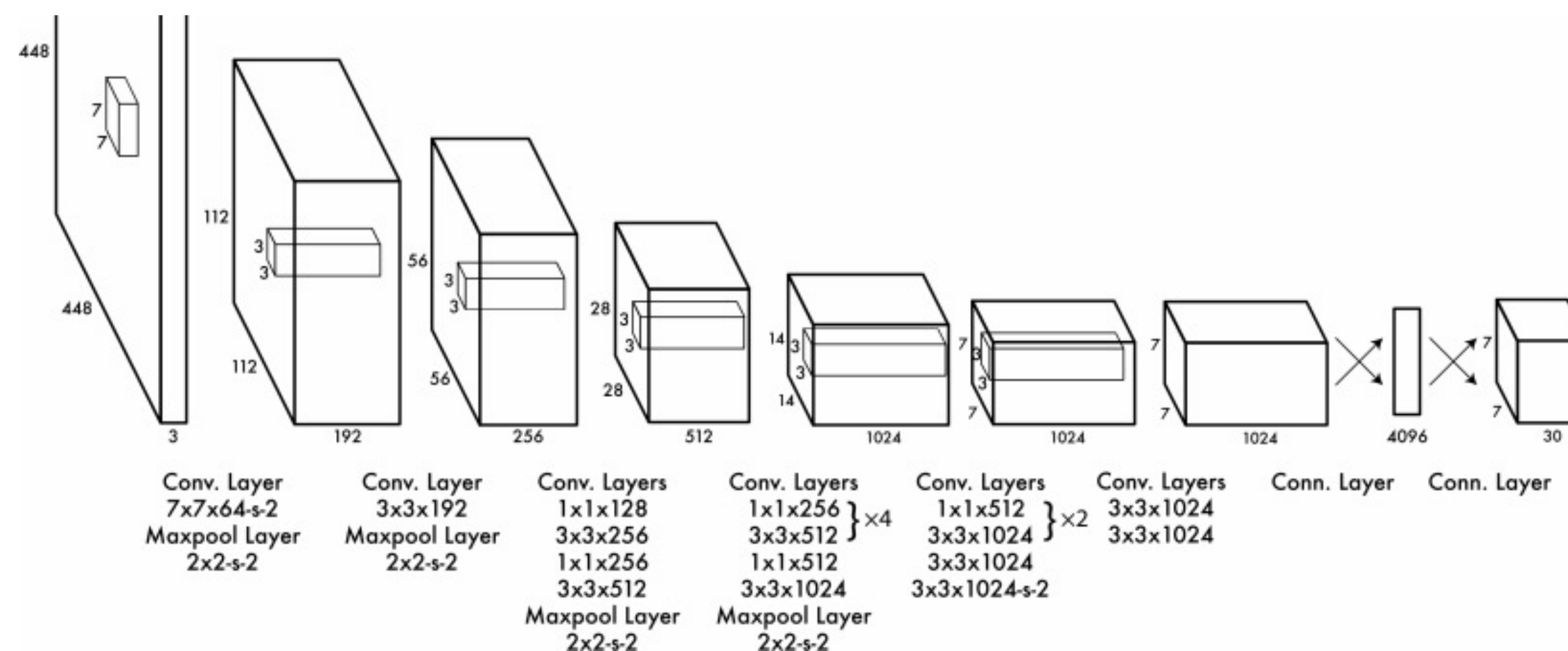
### 基于CNN的一级检测器-YOLO

- You Only Look Once (YOLO) 2015年提出，是深度学习时代的第一个单级检测器。



- YOLOv1将单个神经网络应用于整个图像。该网络将图像分割成多个区域，同时预测每个区域的边界框和概率。

YOLO v1 网络的输入图片尺寸是448\*448，经过多层的卷积池化后，得到一个7\*7\*1024维度的张量，然后再经过两层全连接层，最后输出7\*7\*30维度的张量信息。YOLO v1的网络结果与常规的卷积神经网络的区别在于后端任务网络结构不同，具体而言，对于常规的分类神经网络，其最后的全连接成一般连接一个一维度的向量，向量中的不同元素值代表所属对应类别的概率值，而这里YOLO的输出向量是三维的张量。



### 基于CNN的一级检测器-YOLO

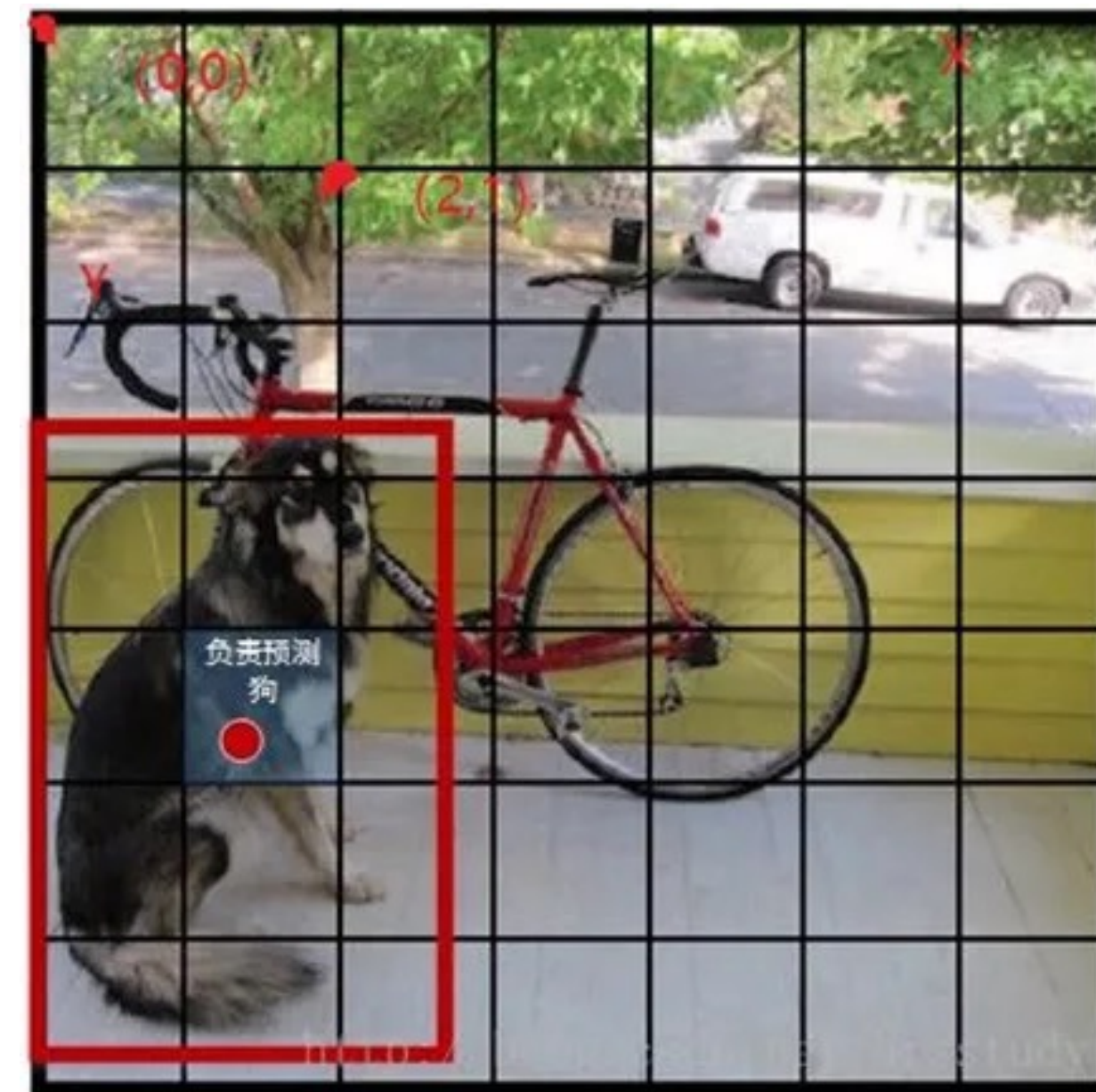
与R-CNN系列算法使用selective search或者RPN网络获取proposal不同，YOLO v1 算法中直接将CNN网络输出的特征图分割成 $S \times S$ 网格，论文中 $S=7$ ，也就是YOLO网络结构图中的倒数第三层维度为 $7 \times 7 \times 30$ 的张量的尺寸，然后每个单元格负责去检测落在该格子内的目标，对于那些没有物体中心点落进来的格子，则不负责预测任何物体。

每个单元格会预测B个边界框（bounding box）以及边界框的置信度（confidence score）。

置信度包含两个方面，一是这个边界框含有目标的可能性大小，二是这个边界框的准确度。前者记为 $\text{Pr}(\text{object})$ ，当该边界框是背景时（即不包含目标），此时 $\text{Pr}(\text{object})=0$ 。而当该边界框包含目标时， $\text{Pr}(\text{object})=1$ 。边界框的准确度可以用预测框与实际框（ground truth）的IOU（intersection over union，交并比）来表征。

以YOLOv1网络结构图中的倒数第三层维度为 $7 \times 7 \times 30$ 的张量为例，这里的 $30 = (4+1) \times 2 + 20$ ，其中4表示矩形框的中心点坐标 $x, y$ ，以及长度 $h$ 和宽度 $w$ ，1是检测物体的置信度 $c$ ；2是论文中设定的每个格子所回归的两个矩形框，这个参数也可以更改；20表示预测20个类别，因此张量的形状为 $7 \times 7 \times 30$ 。

注意，论文中设定的每个格子回归的矩形框的个数为2，可以更改，但是不管设定回归几个矩形框，最后只会选择置信度更大的矩形框来作为输出，也就是说每个格子只会输出一个预测矩形框，只能预测一个物体。论文中设定的 $7 \times 7$ 的格子，也就是说该网络最多可以预测49个物体。



优点：

- 相比于两阶段的检测算法，YOLO的检测速度非常快；

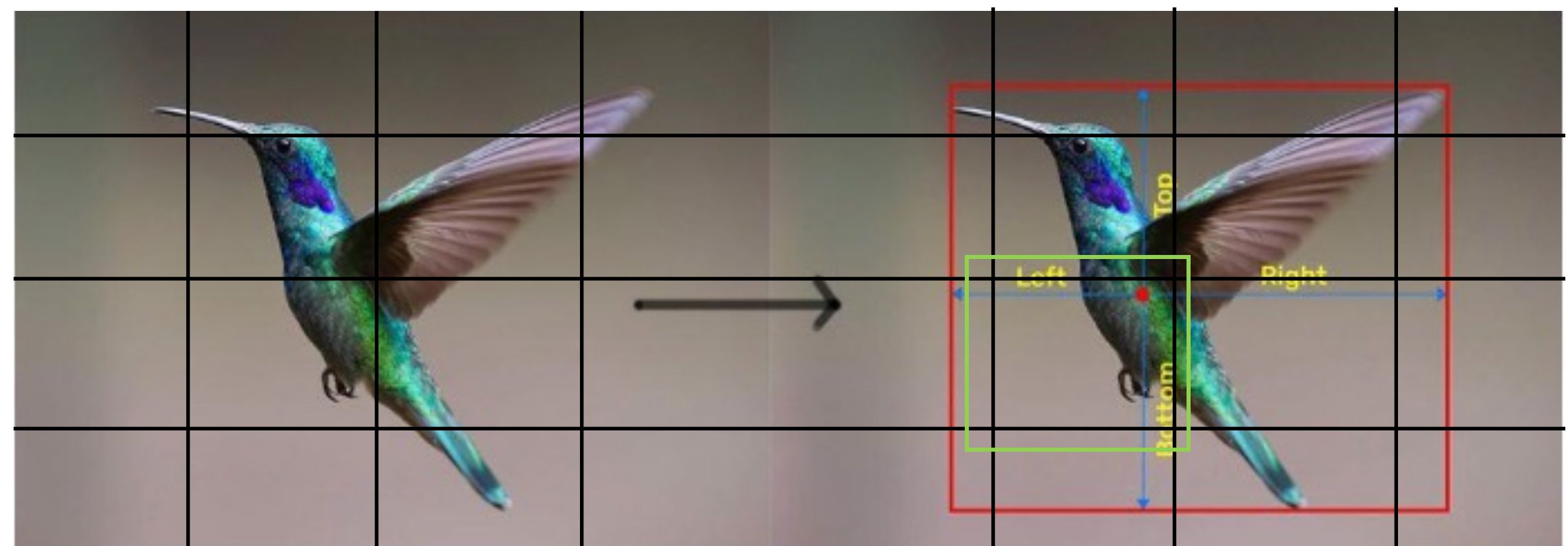
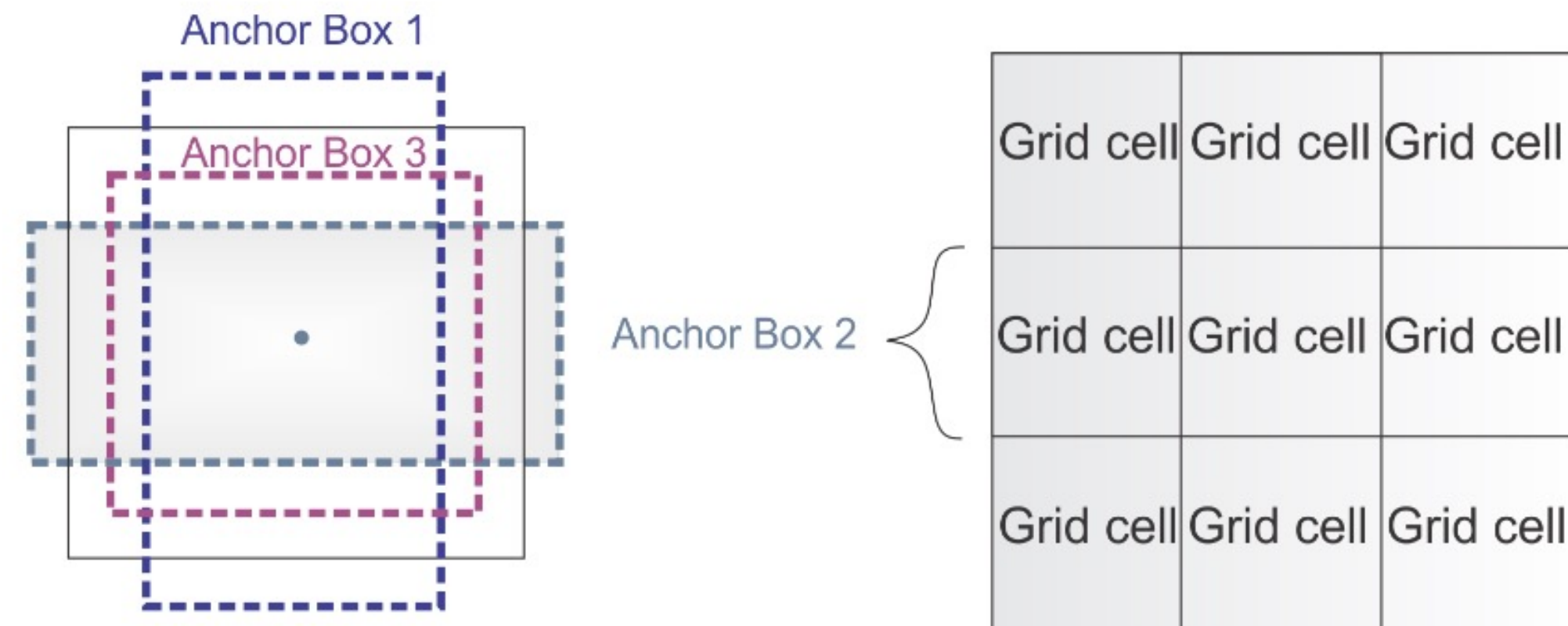
缺点：

- 问题存在定位不准以及召回率不如基于region proposal方法的；
- 对距离很近的小物体检测效果不好。

## 二、目标检测模型

### Yolo系列 后续改进

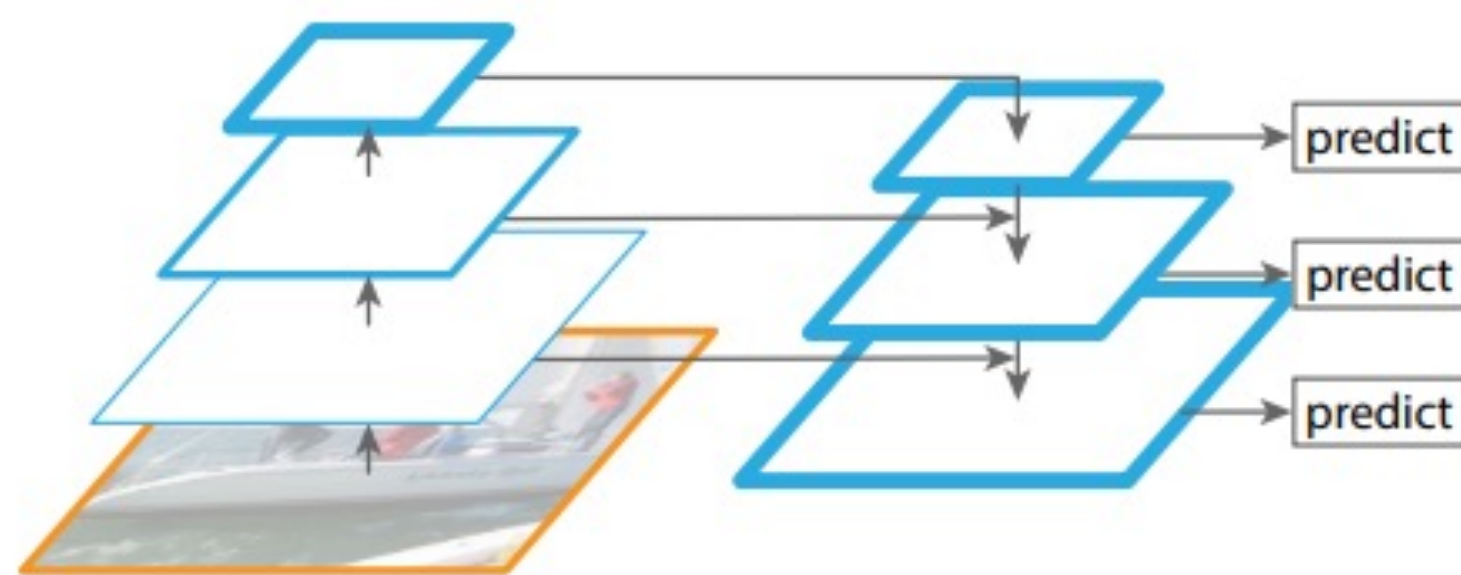
- 仿照Faster-RCNN，使用anchor预测边界框，使用的anchor的大小来自于VOC和COCO数据集的统计
- 不再使用anchor——anchor free
- 目标的中心点只在一个Grid cell，因此只有这一个Grid Cell负责预测目标与中心点的偏差



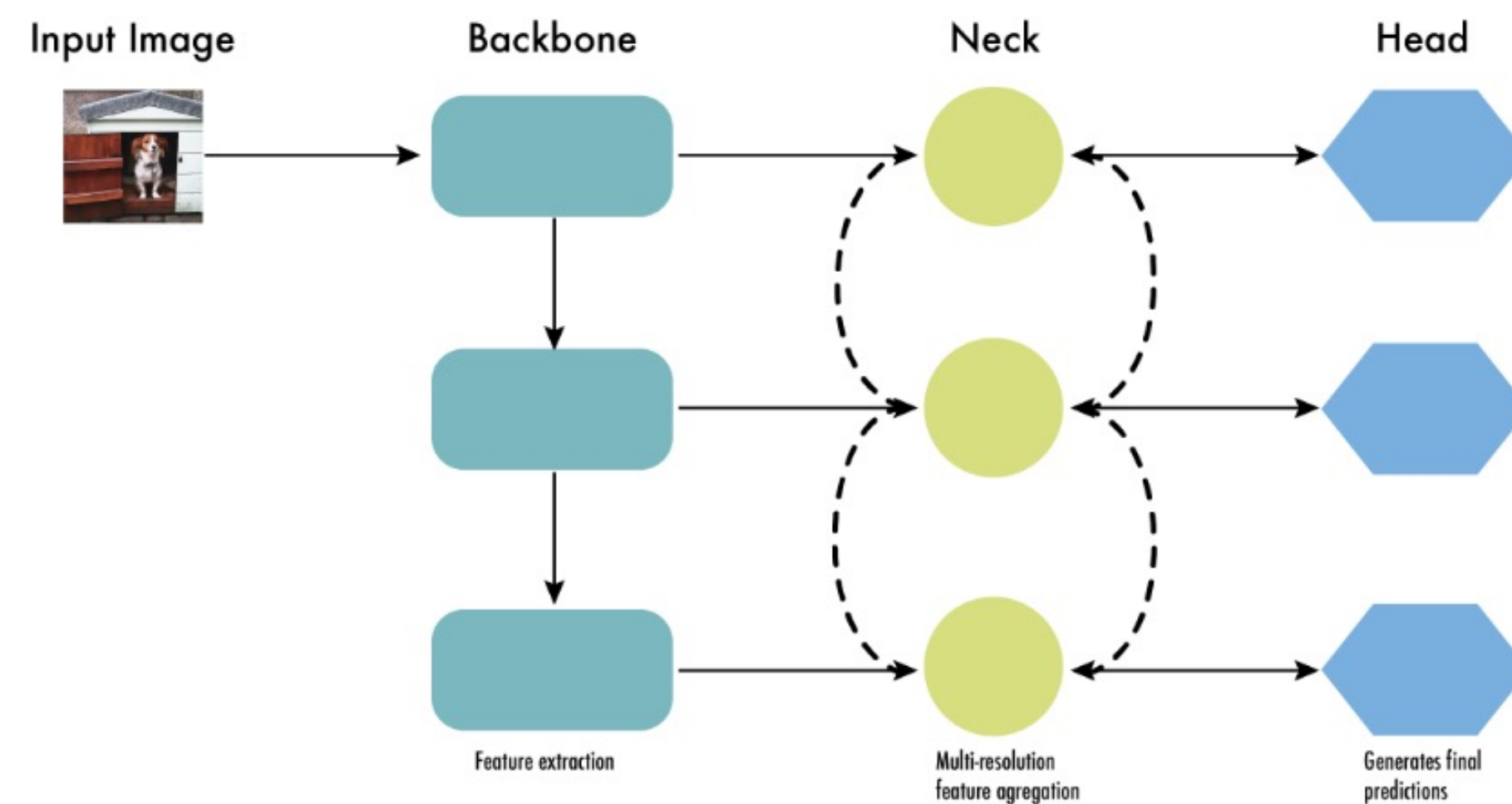
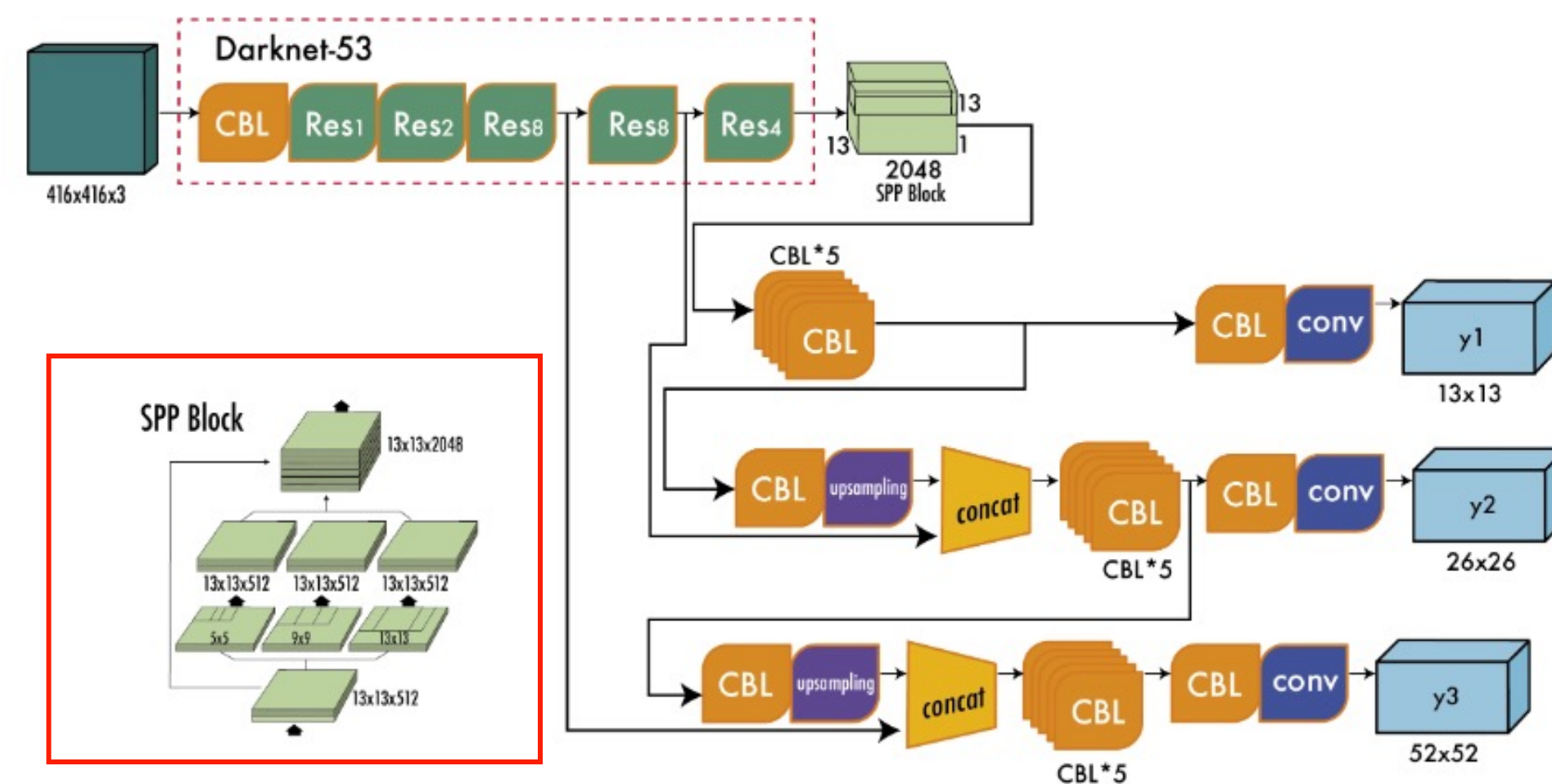
## 二、目标检测模型

### Yolo系列 后续改进

- 类似于Feature Pyramid Networks，引入多尺度预测



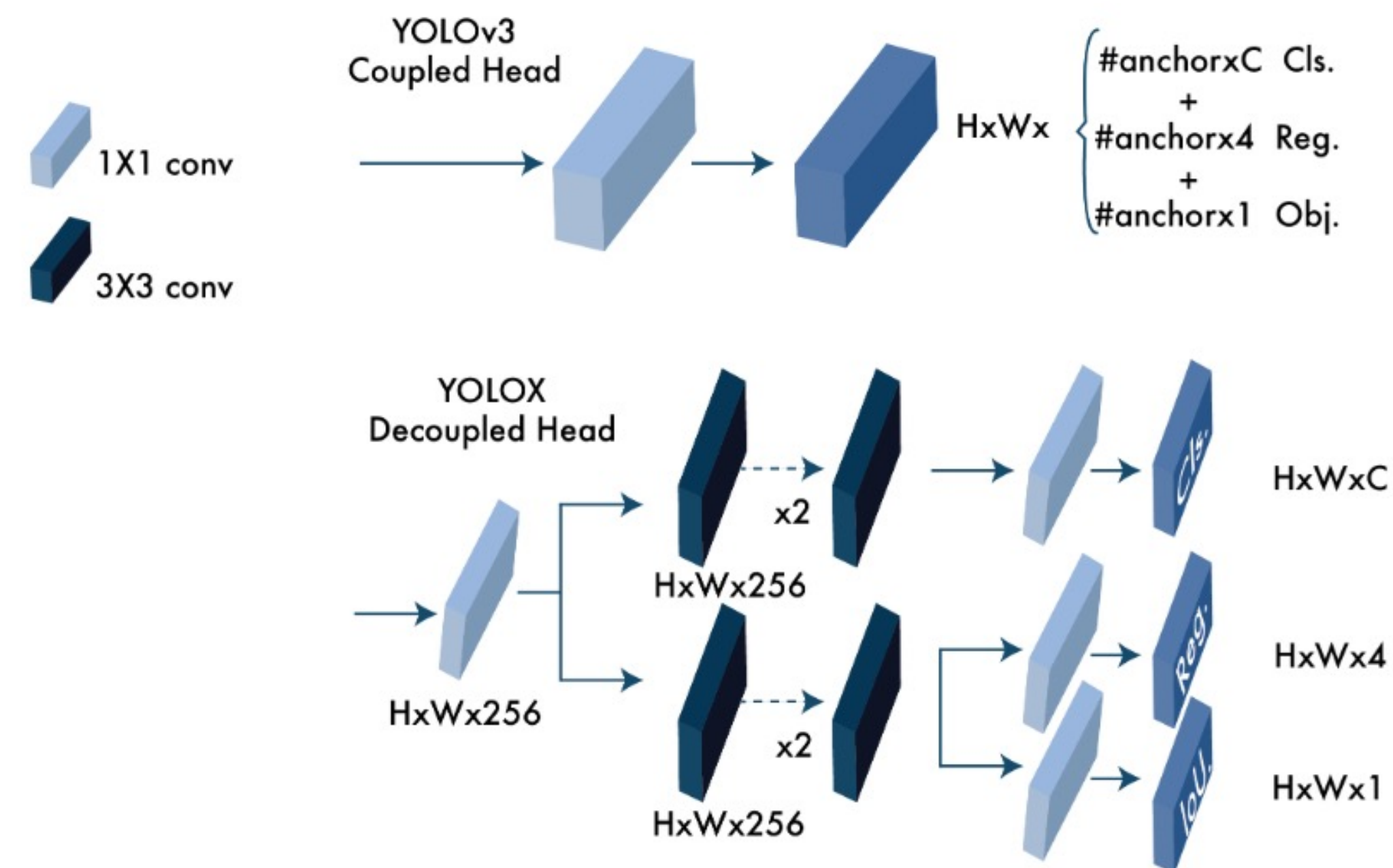
- 使用空间金字塔池化(Spatial pyramid pooling, SPP)



## 二、目标检测模型

### Yolo系列 后续改进

- 预测头从一个耦合头变为解耦头
- Coupled Head：同时预测类别（分类任务），位置（回归任务）
- Decoupled Head：将类别（分类任务）与位置（回归任务）的预测解耦，使用不同的网络进行



## 二、目标检测模型

### 基于CNN的一级检测器-SSD

- SSD和Yolo一样都是采用一个CNN网络来进行检测，但是却采用了多尺度的特征图，这大大提高了单级检测器的检测精度，特别是对于一些小目标。SSD在检测速度和准确度上都有优势。其基本架构如图1所示。下面将SSD核心理念总结为以下三点：

#### 1).采用多尺度特征图用于检测

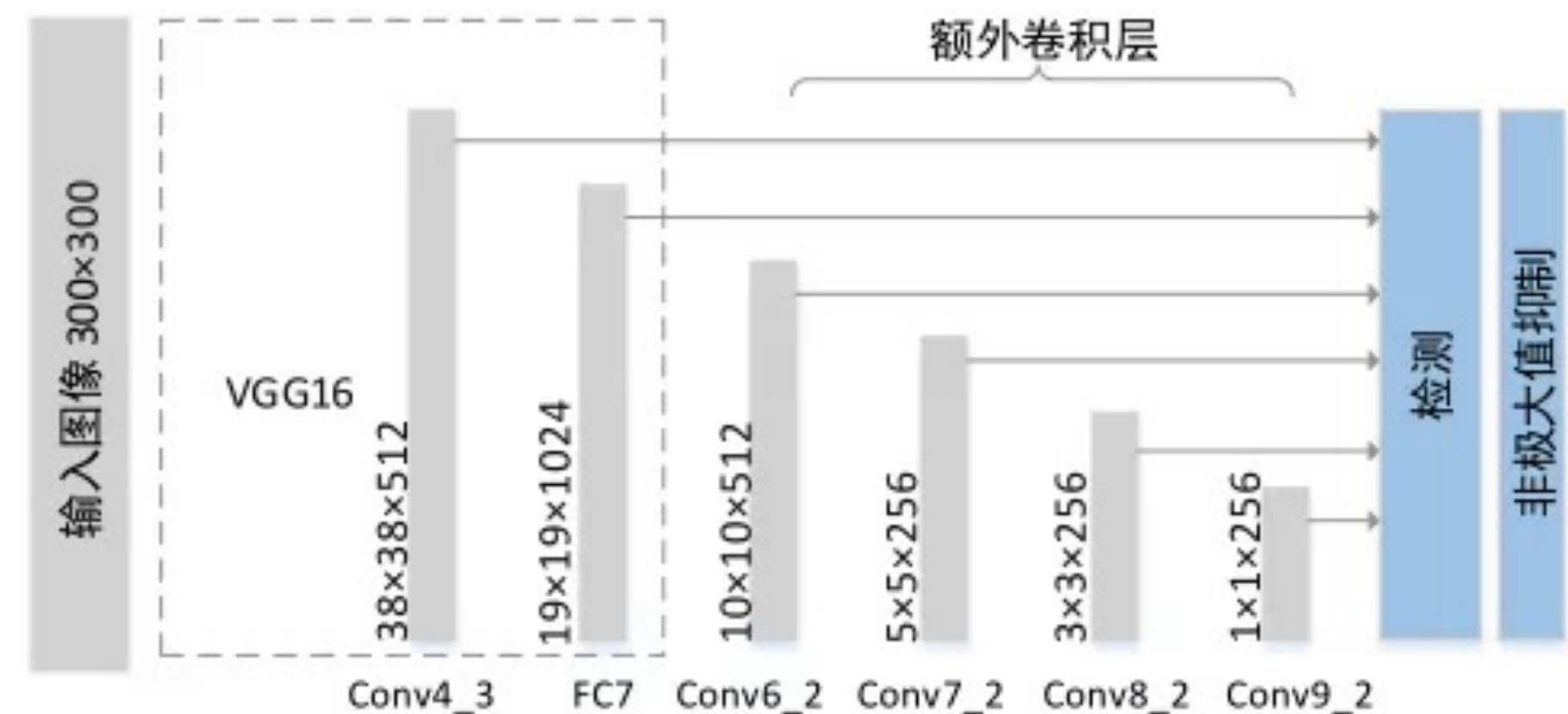
SSD 网络结构采用 VGG16 作为基础模型，然后在 VGG16 的基础上新增了卷积层来获得更多的特征图用于检测。主要改变是将第六层（FC6）第七层（FC7）的全连接层转换成两个卷积层，并在第七层（FC7）之后额外增加了四个卷积层，SSD 网络结构如图 1 所示。预测过程中，在 Conv4\_3、FC7 及新增的四个卷积层所产生的六个特征图进行目标类别与位置的预测。

#### 2).采用卷积进行检测

与Yolo最后采用全连接层不同，SSD直接采用卷积对不同的特征图来进行提取检测结果。具体原因是，全连接层会破坏特征的空间结构。具体做法是使用3\*3的卷积进行位置、类别的预测，然后使用reshape缩放维度，可参考SSD源码中的multibox\_layer函数。

#### 3).设置先验框

Yolo中，一张图片经过backbone进行特征提取后，生成一个宽高为7\*7的feature map，也就是49个网格，然后对每个网格预测类别、置信度、坐标等信息，这样的做法使得召回率较低。SSD中借鉴了Faster R-CNN中anchor的理念，设置不同尺度或者不同长宽比的先验框，预测框有七千多个，因此在很大程度上解决了召回率低的问题。



### 后续改进——目标检测和语义分割

- 通过分割提高目标检测的主要方法有两种：
- 1) 采用丰富的特征学习；
- 2) 采用多任务损失函数学习。

特征学习

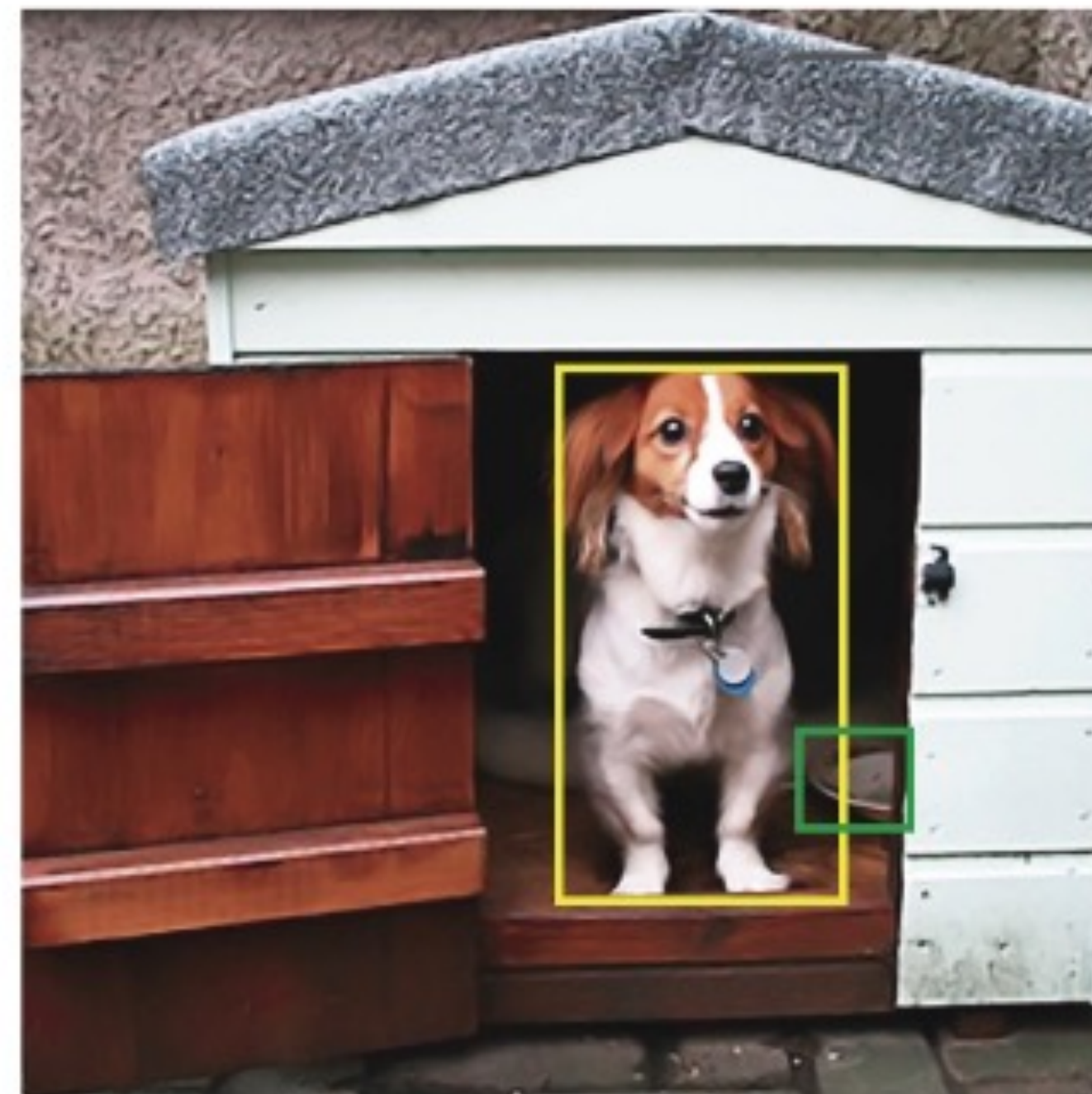
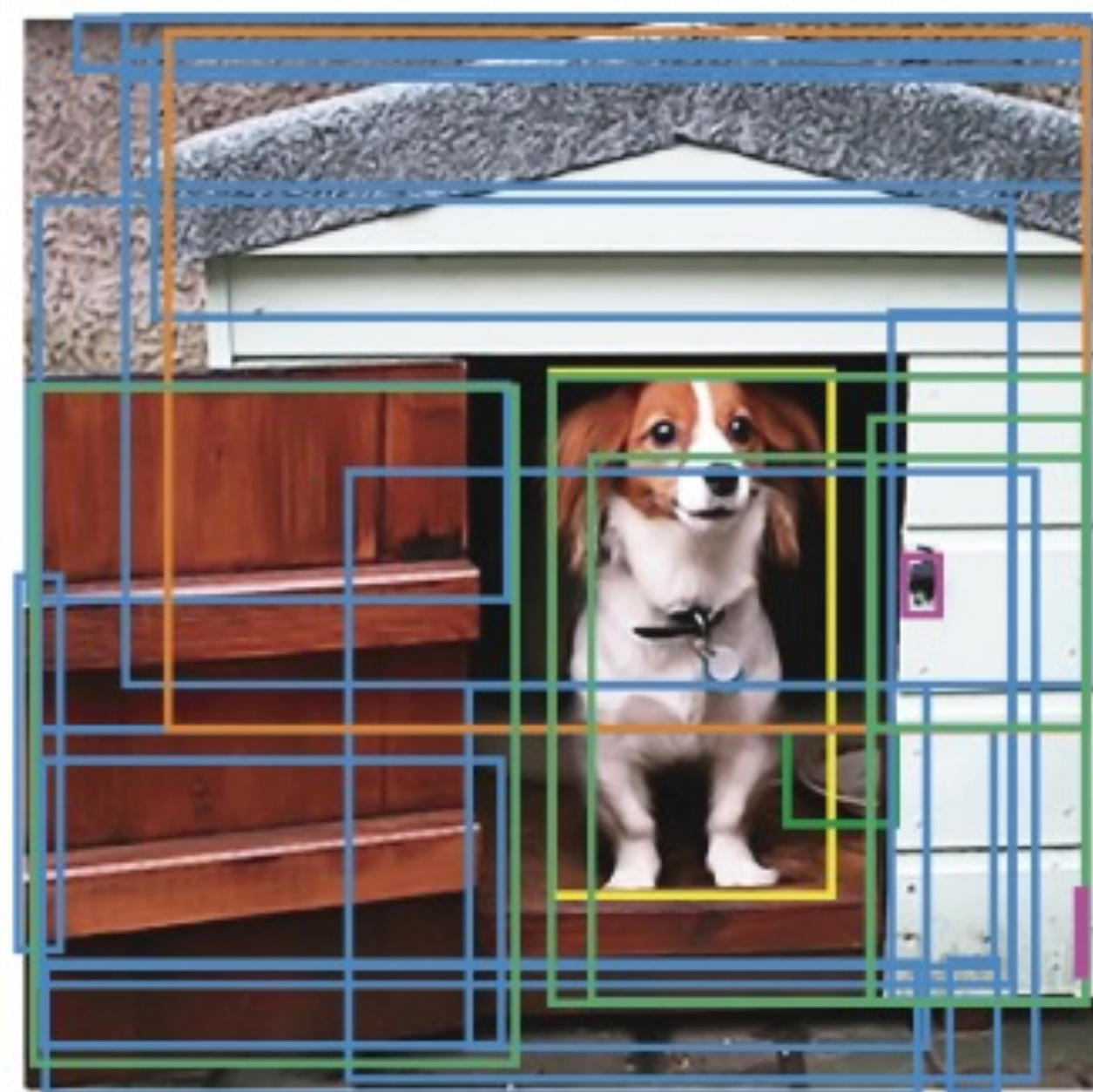
损失函数

最简单的方法是将分割网络看作一个固定的特征提取器，并将其作为附加特征集成到检测框架中。该方法的优点是易于实现，缺点是分割网络可能带来额外的计算。

另一种方法是在原有检测框架的基础上引入额外的分割分支，用多任务损失函数（分割损失+检测损失）训练该模型。在大多数情况下，分割分支将在推理阶段被删除。优点是检测速度不受影响，缺点是训练需要像素级的图像标注。为此，一些研究人员采用了“弱监督学习”的思想：他们不是基于像素级注释掩码进行训练，而是基于边界框级注释训练分割分支

## 二、目标检测模型

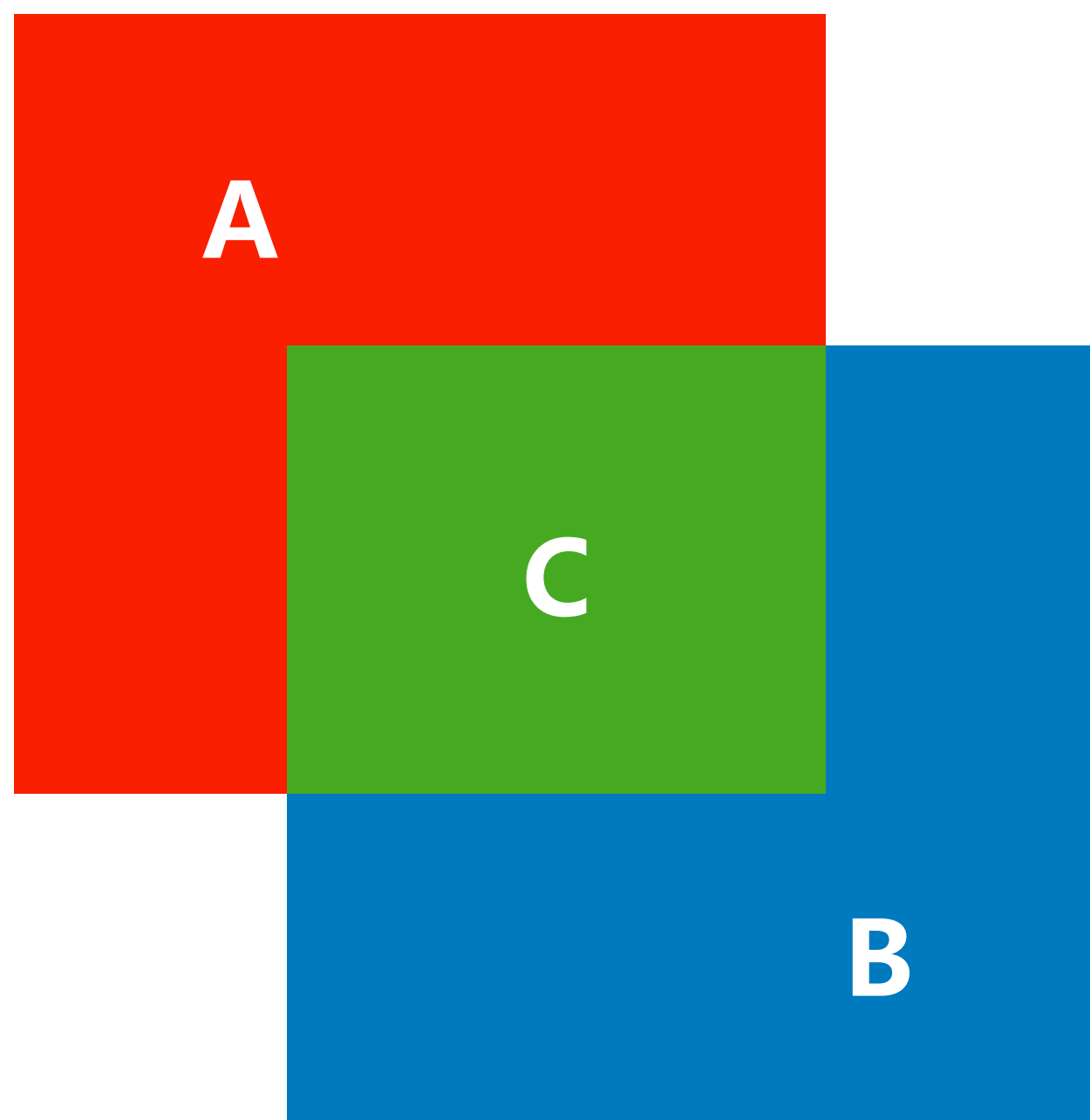
### 检测后处理方法——非极大值抑制NMS



设置一个IoU threshold，计算每两个框之间的IoU，对于超过IoU threshold的框，只保留置信度最高的框

## 检测评价指标——IoU

**IoU:** 模型预测的bbox和Ground Truth之间的交并比



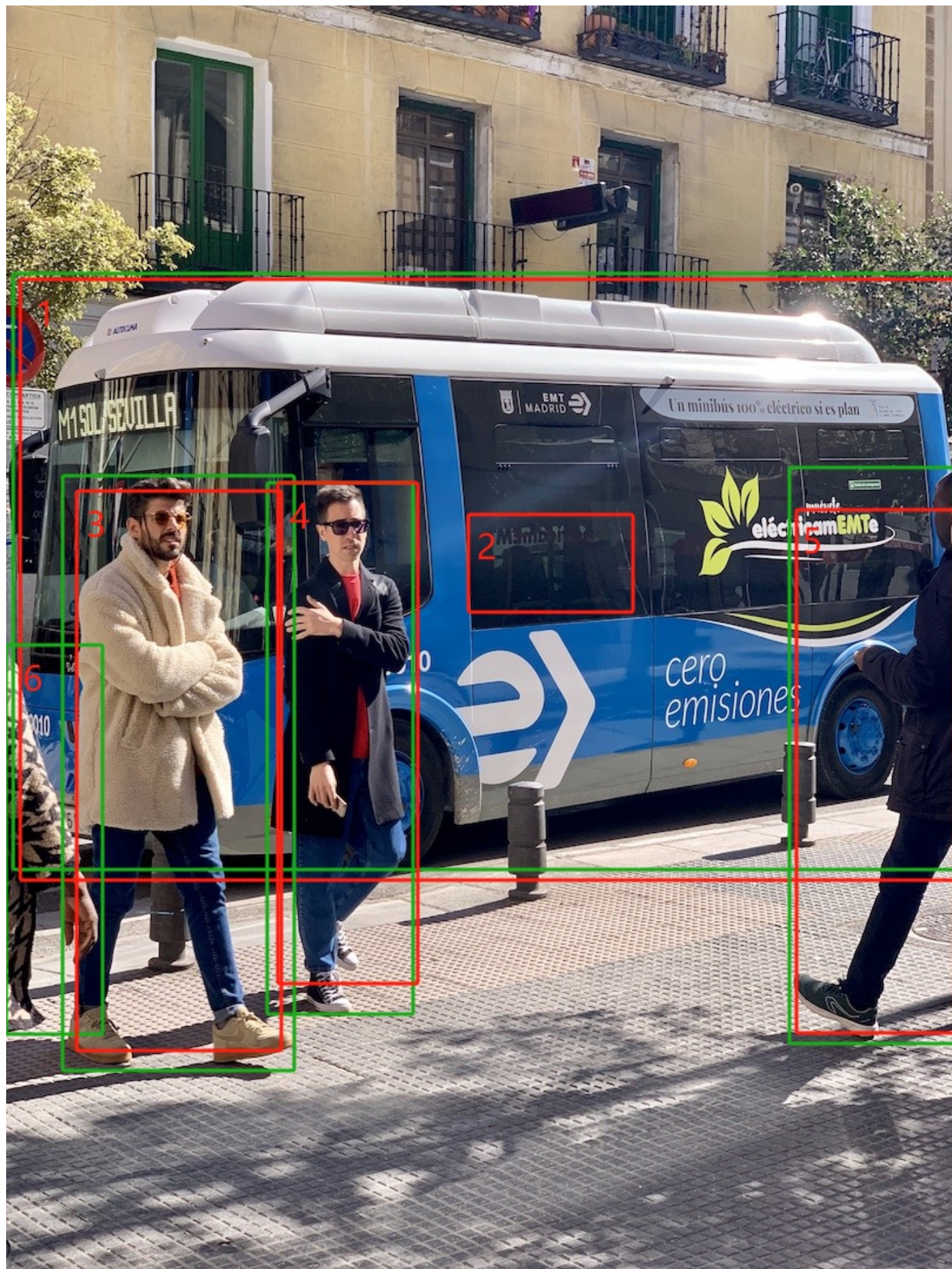
$$\text{IoU} = \frac{A \cap B}{A \cup B}$$

$$A \cup B = A + B - C$$

$$A \cap B = C$$

## 三、检测结果评估

### 检测评价指标——TP、FP、FN



**TP(True Positive):**  $IoU > IoU \text{ threshold}$ 的检测框数量 (同一GT只计算一次)

**FP(False Positive):**  $IoU \leq IoU \text{ threshold}$ 的检测框数量 (或是检测到同一个GT的多余检测框的数量)

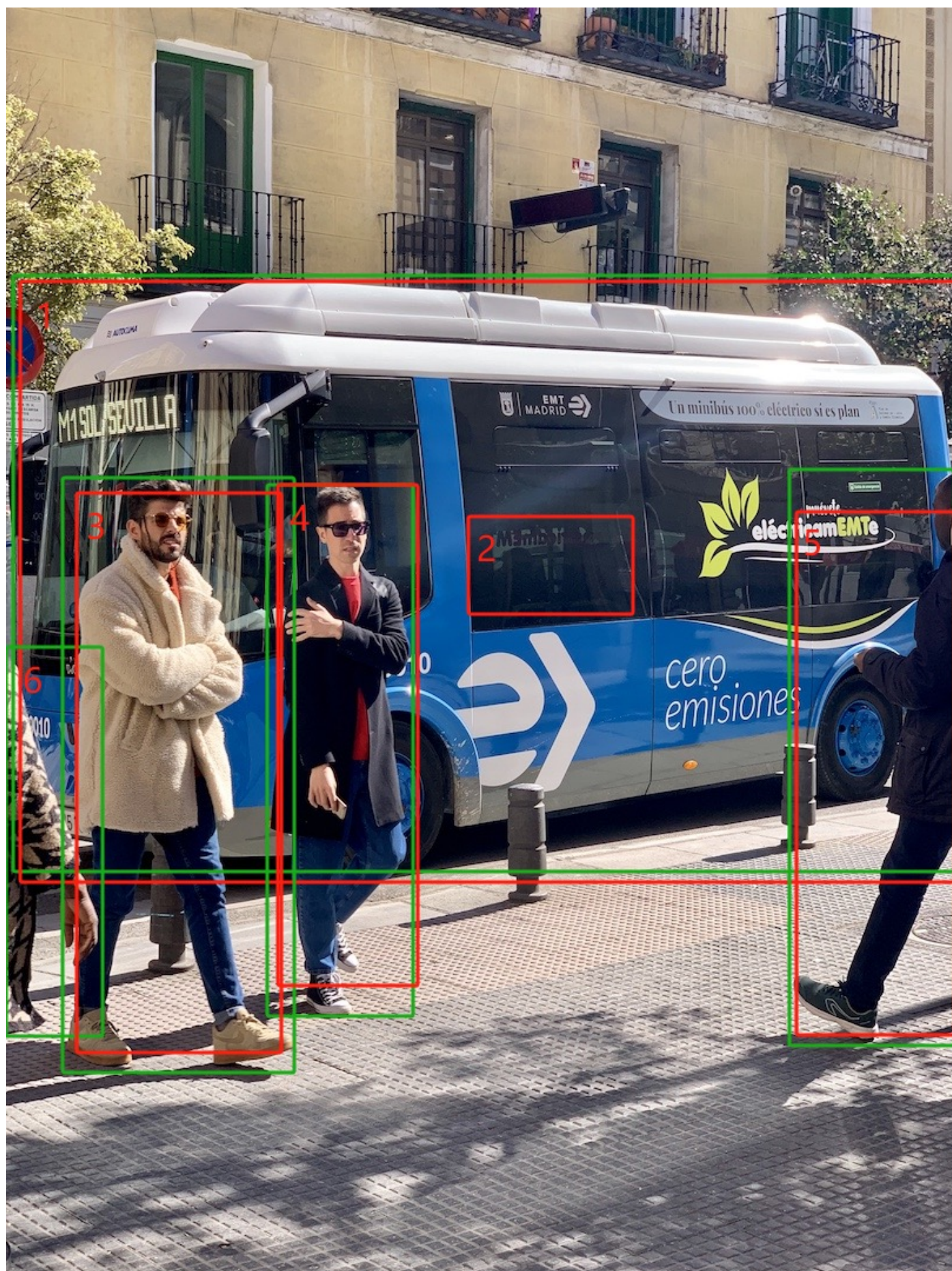
**FN(False Negative):** 没有检测到的GT的数量



- IoU threshold为0.5
- Box 1为TP (与GT的IoU超过0.5)
- Box 2为FP (与GT的IoU不足0.5)
- Box 3为TP (与GT的IoU超过0.5)
- Box 4为TP (与GT的IoU超过0.5)
- Box 5为TP (与GT的IoU超过0.5)
- Box 6为FN (未检测到的GT)

## 三、检测结果评估

### 检测评价指标——Precision(查准率)、Recall(查全率)



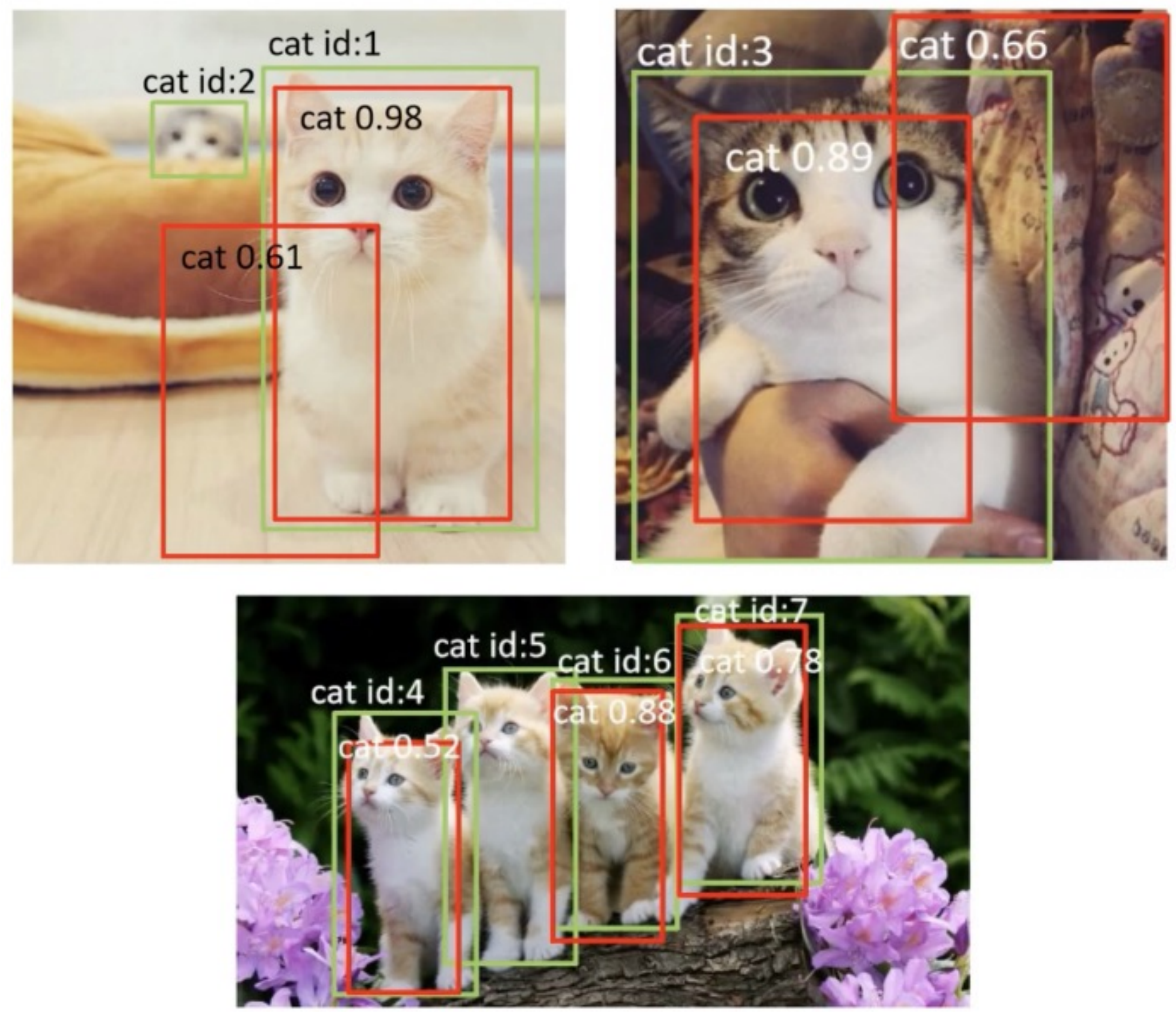
- **Precision (查准率)** :  $TP / (TP + FP)$  模型预测的所有目标中，预测正确的比例
- **Recall (查全率)** :  $TP / (TP + FN)$  所有真实目标中，模型预测正确的目标比例

- IoU threshold=0.5 , TP=4 , FP=1 , FN=1

- $Precision = \frac{TP}{TP + FP} = \frac{4}{5} = 0.8$

- $Recall = \frac{TP}{TP + FN} = \frac{4}{5} = 0.8$

## 检测评价指标——AP、mAP



绿色框为GT 红色框为模型检测结果

**AP:** Average Precision 对单个类别P-R曲线下的面积  
**mAP:** mean Average Precision, 即各类别AP的平均值

num\_ob = 7

| GT ID | Confidence | OB(IOU=0.5) |
|-------|------------|-------------|
| 1     | 0.98       | True        |
| 3     | 0.89       | True        |
| 6     | 0.88       | True        |
| 7     | 0.78       | True        |
| 3     | 0.66       | False       |
| 1     | 0.61       | False       |
| 4     | 0.52       | True        |

# 三、检测结果评估

## 检测评价指标——AP、mAP

按顺序（每个框的Confidence）进行计算

num\_ob = 7

| GT ID | Confidence | OB(IOU=0.5) |
|-------|------------|-------------|
| 1     | 0.98       | True        |
| 3     | 0.89       | True        |
| 6     | 0.88       | True        |
| 7     | 0.78       | True        |
| 3     | 0.66       | False       |
| 1     | 0.61       | False       |
| 4     | 0.52       | True        |

confidence=0.98  
TP=1; FP=0;  
FN=6

confidence=0.89  
TP=2; FP=0;  
FN=5

confidence=0.88  
TP=3; FP=0;  
FN=4

confidence=0.78  
TP=4; FP=0;  
FN=3

confidence=0.66  
TP=4; FP=1;  
FN=3

confidence=0.61  
TP=4; FP=2;  
FN=3

confidence=0.52  
TP=5; FP=2;  
FN=2

| Rank | Precision | Recall |
|------|-----------|--------|
| 1    | 1.0       | 0.14   |
| 2    | 1.0       | 0.28   |
| 3    | 1.0       | 0.42   |
| 4    | 1.0       | 0.57   |
| 5    | 0.80      | 0.57   |
| 6    | 0.66      | 0.57   |
| 7    | 0.71      | 0.71   |

检测框的数量=7

GT的数量=7

以Recall为横坐标，因此  
Recall相同时只保留  
Precision最大值

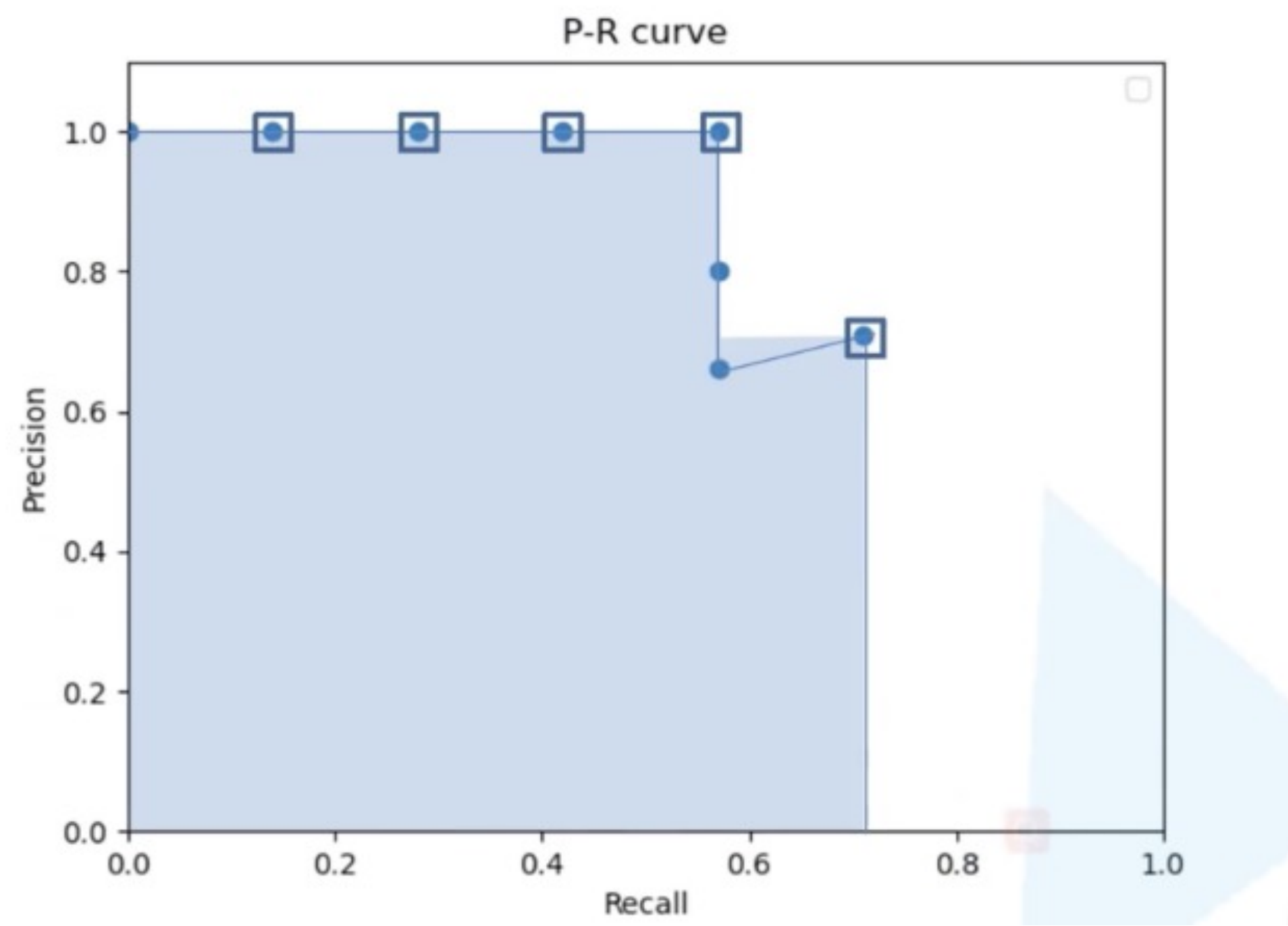


# 三、检测结果评估

## 检测评价指标——AP、mAP

| Rank | Precision | Recall |
|------|-----------|--------|
| 1    | 1.0       | 0.14   |
| 2    | 1.0       | 0.28   |
| 3    | 1.0       | 0.42   |
| 4    | 1.0       | 0.57   |
| 5    | 0.80      | 0.57   |
| 6    | 0.66      | 0.57   |
| 7    | 0.71      | 0.71   |

Precision-Recall曲线



mAP@0.5: IoU=0.5时的mAP  
mAP@0.75: IoU=0.75时的mAP  
mAP@0.5:0.95 : 0.5-0.95共10个值，分别计算mAP，求均值

计算P-R曲线围成的面积

$$AP = (0.14 - 0) \times 1.0 + (0.28 - 0.14) \times 1.0 + (0.42 - 0.28) \times 1.0 + (0.57 - 0.42) \times 1.0 + (0.71 - 0.57) \times 0.71 = 0.6694$$

$$mAP = \frac{AP1+AP2+\dots}{N} \quad \text{多个类别的情况}$$

## 行人检测

### 困难与挑战

- 行人检测作为一种重要的对象检测应用，已在自动驾驶，视频监控，犯罪侦查等许多领域受到广泛关注。
- 行人检测的挑战和困难可以概括如下：小行人、硬底片、拥挤的行人、实时检测



### 检测方法

- 行人检测有一个很长的研究历史，它的发展可以分为两个技术时期：
  - 1) 传统的行人检测
  - 2) 基于深度学习的行人检测。
- 传统行人检测方法有：改善小型行人检测、改善硬阴性检测、改善密集和封闭的行人检测

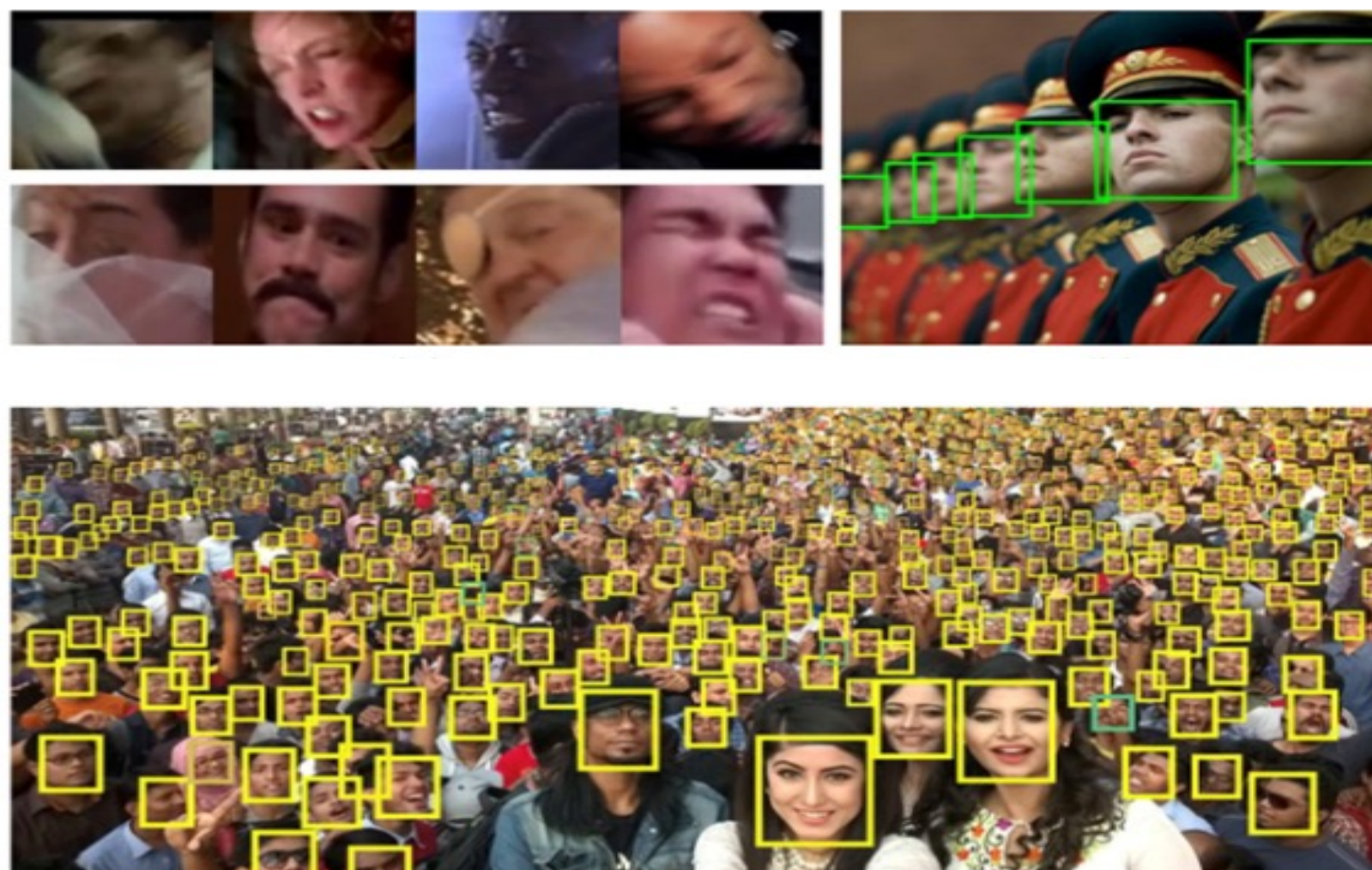


## 四、检测应用

### 人脸检测

#### 困难与挑战

- 人脸可以呈现出多种表情、肤色、姿势和动作；
- 人脸可能被其他物体遮挡部分；
- 检测多种尺度下的人脸，特别是一些微小的人脸；
- 移动设备上的人脸检测通常需要CPU的实时检测速度。



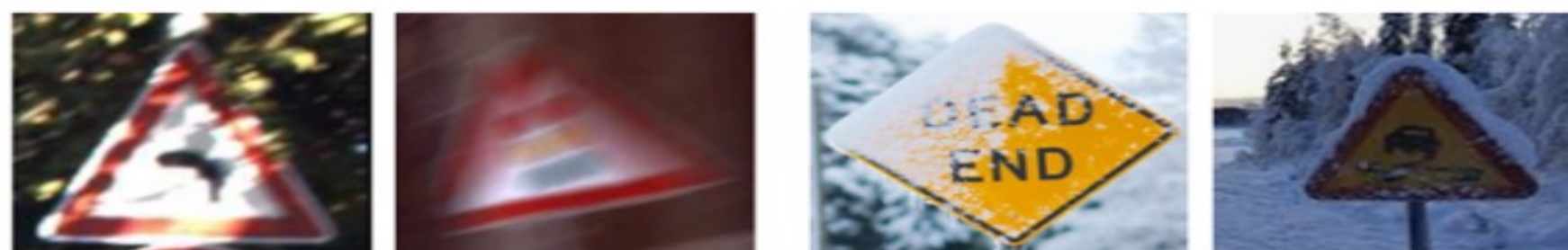
#### 检测方法

- 早期的人脸检测算法可以分为三组：1) 基于规则的方法。2) 基于子空间分析的方法。3) 基于学习的方法
- 在深度学习时代，大多数人脸检测算法都遵循一般的目标检测思想，如 Faster RCNN 和 SSD。有几种方法：
- 级联检测是深度学习时代加速人脸检测最常用的方法。
- “人脸定标（face calibration）”的思想通过估计定标参数或通过多个检测阶段的渐进定标来改进多姿态人脸检测。
- 近年来多尺度人脸检测的研究采用了与一般目标检测相似的检测策略，包括多尺度特征融合和多分辨率检测

## 交通标志和交通灯的自动检测

### 困难与挑战

- 灯光变动：当车辆行驶在强光下或夜间时，检测尤为困难。
- 动态模糊：由于汽车的运动，车载摄像头拍摄到的图像会变得模糊。
- 恶劣天气：在恶劣天气下，如雨雪天气，图像质量会受到影响。
- 实时检测：这对自动驾驶特别重要。



### 检测方法

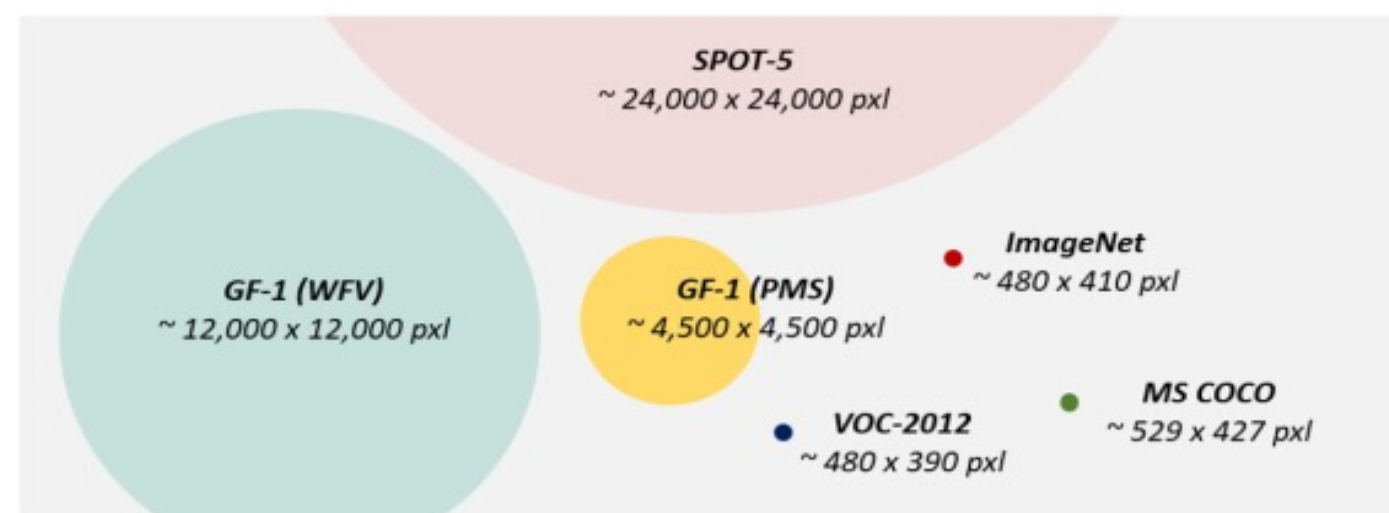
- 传统的检测方法通常基于颜色阈值，视觉显著性检测，形态滤波，边缘/轮廓分析。一些研究者还探索了除了基于视觉的方法之外的其他解决方案，例如将 GPS 与数字地图相结合用于交通灯检测。
- 在深度学习时代，一些著名的检测器如 Faster RCNN 和 SSD 被应用到交通标志/灯光检测任务中。在这些检测器的基础上，一些新的技术，如注意机制和对抗性训练被用来改善复杂交通环境下的检测。



## 遥感目标检测

### 困难与挑战

- 由于遥感图像数据量巨大，如何快速准确地检测遥感目标仍然是一个问题。
- 地球表面每天有50%以上被云覆盖，目标可能被遮挡。
- 不同传感器（例如，不同的调制和分辨率）拍摄的遥感图像存在很大的差异。



### 检测方法

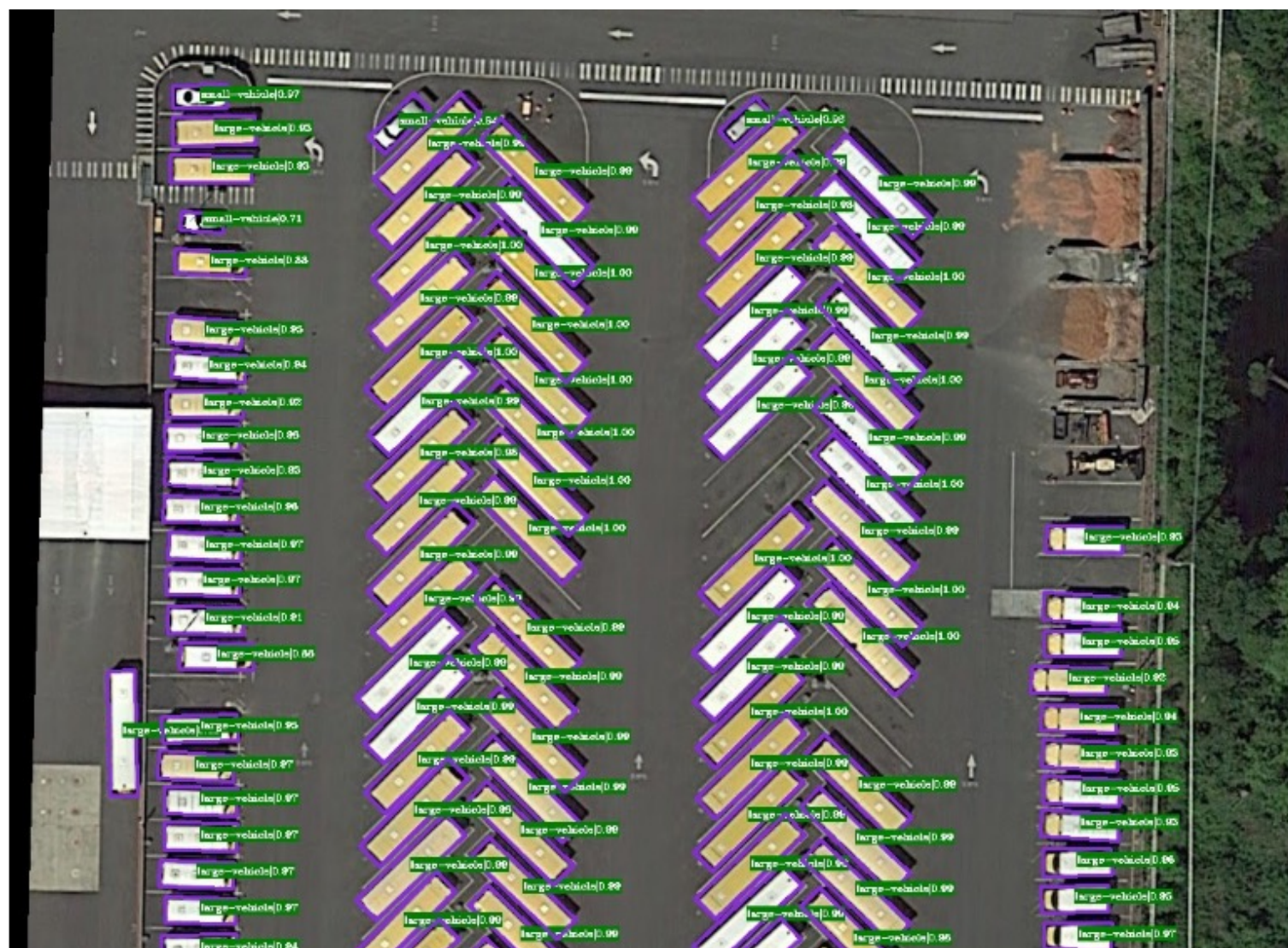
- 传统的遥感目标检测方法大多采用两阶段检测范式：1) 候选提取；2) 目标验证。为了检测具有特定结构和形状的目标，如油罐和近岸船舶，需要使用一些领域知识。例如，油罐检测可以看作是圆/弧检测问题。近岸船舶检测可视为前甲板和船尾的检测。
- RCNN在2014年取得巨大成功后，一般目标检测框架像 Faster RCNN 和 SSD 用在了遥感领域。为了检测不同方向的目标，一些研究人员改进了ROI池化层，以获得更好的旋转不变性。为了提高区域自适应能力，一些研究者从贝叶斯的角度提出了检测方法，在检测阶段，根据测试图像的分布自适应更新模型。此外，注意机制和特征融合策略也被用来改进小目标检测。



## 旋转目标检测

### 困难与挑战

- 在自然场景图像的目标检测中，通常使用水平四边形框作为检测框对目标的位置进行框定。但是，在遥感目标检测以及文字检测等相关场景，目标通常情况都是倾斜的。
- 如何确定旋转方向
- 目标密集，且有大量小目标



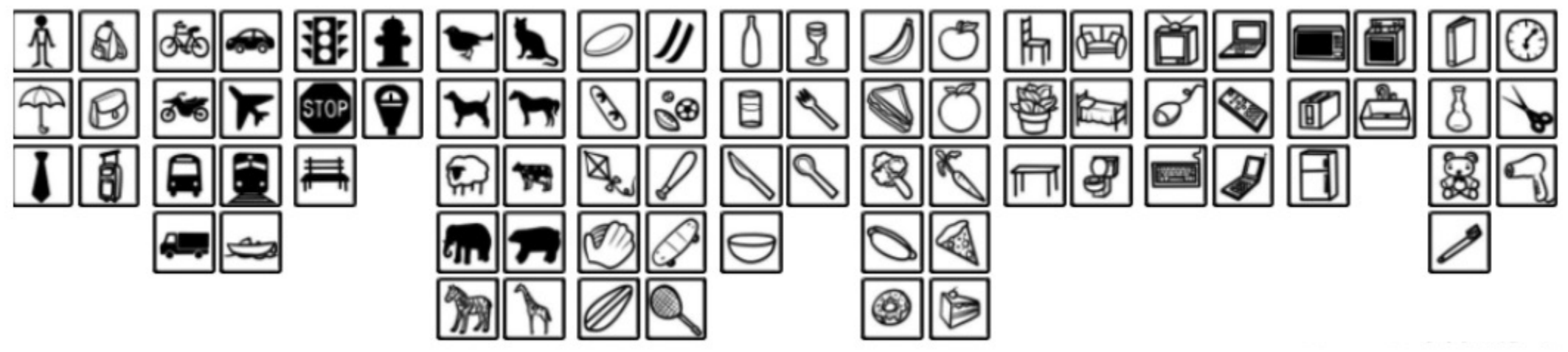
### 检测方法

- 在检测头中添加一个预测角度的分支
- 在生成锚框时考虑不同的角度，使网络能够直接预测目标物体的旋转角度
- 使用注意力机制来增强模型对旋转目标的敏感性。
- 综合使用多个尺度和角度的特征来提高检测的准确性。

# 五、目标检测数据集

## COCO(Common Objects in Context)

- 是一个大规模的，适用于目标检测，图像分割，Image Captioning任务的数据集
- 目标检测部分共有91类，有328,000张图片，2,500,000个标注



```
{
  "info": info,
  "images": [image], //列表
  "annotations": [annotation], //列表
  "categories": [category], //列表
  "licenses": [license], //列表
}

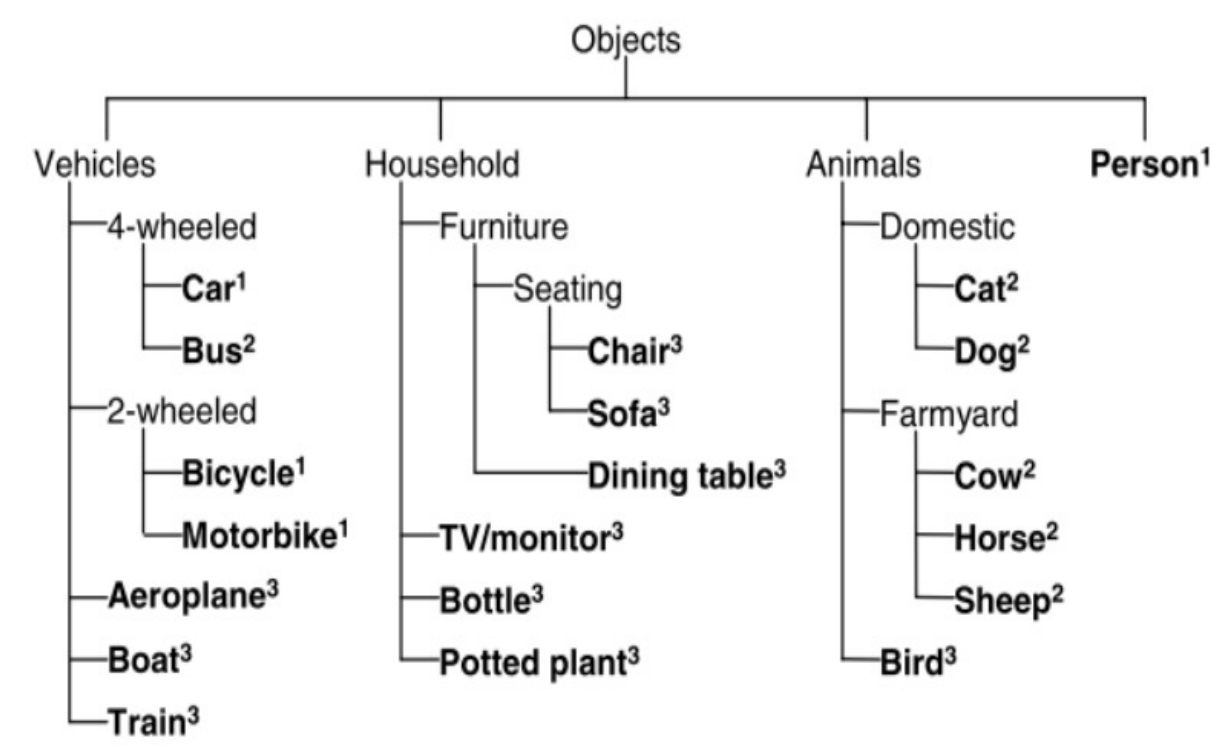
category{
  "id": int, //类别序号
  "name": str, //类别名称
  "supercategory": str, //父类
}

annotation{
  "id": int, //标注id
  "image_id": int, //所属图像id
  "category_id": int, //类别id
  "segmentation": RLE or [polygon], //图像分割标注
  "area": float, //区域面积
  "bbox": [x,y,width,height], //目标框左上角坐标以及宽
  "iscrowd": 0 or 1, //是否密集
}
```

高

## PASCAL VOC(Visual Object Classes)

- 来源于PASCAL&VOC挑战赛
- 是一个大规模的，适用于目标检测的数据集，共包含20个类别
- VOC2007trainval共5011张图片，VOC2007test共4952张图片，VOC2012trainval共11540张图片



```
<annotation>
  <folder>VOC</folder>          # 图像所在文件夹
  <filename>000032.jpg</filename> # 图像文件名
  <source>                        # 图像源
    <database>The VOC Database</database>
    <annotation>PASCAL VOC</annotation>
    <image>flickr</image>
  </source>
  <size>                          # 图像尺寸信息
    <width>500</width>    # 图像宽度
    <height>281</height> # 图像高度
    <depth>3</depth>    # 图像通道数
  </size>
  <segmented>0</segmented> # 图像是否用于分割，0代表不适用，对目标检测而言没
关系
  <object>                      # 一个目标对象的信息
    <name>aeroplane</name>    # 目标的类别名
    <pose>Frontal</pose>      # 拍摄角度，若无一般为Unspecified
    <truncated>0</truncated>  # 是否被截断，0表示完整未截断
    <difficult>0</difficult>  # 是否难以识别，0表示不难识别
    <bndbox>                  # 边界框信息
      <xmin>104</xmin>    # 左上角x
      <ymin>78</ymin>    # 左上角y
      <xmax>375</xmax>    # 右下角x
      <ymax>183</ymax>    # 右下角y
    </bndbox>
  </object>
  <object>
其他object信息，这里省略
  </object>
</annotation>
```